

CRITICAL READER'S MEMORANDUM

The Harbor After the Harbor

What the coordination papers are almost arriving at - and how to build the institution hiding inside them

Manuscript critique

Complete exercise solution key

Local effect-control runtime

Remote project institution

Mutually confidential Sealed Harbor

Prepared for Erich Owens

August 2026

Contents

This report is organized as argument first, solution key second, and architecture program third.

Reader's note	6
Executive verdict	6
What is alive	6
What is obscuring it	6
The thesis you are almost stating	7
Product consequence	7
What deserves lavish development	8
1. Claim discipline as the epistemic kernel	8
2. Attention as control, not dashboard aesthetics	8
3. The principal above the actor	8
4. The institution that outlives the repo	9
5. Mutually confidential computation	9
What is not yet interesting	9
Correctness audit: claims that must change	10
Chapter I: Signal Detection Theory points the wrong way	10
Chapter I: roadmap-as-truth repeats the Scott failure	10
Chapter II: exact-key uniqueness is not mutual exclusion	10
Chapter II: WAL auto-checkpoint does not bound elapsed loss time	10
Chapter II: "reference monitor" overstates the same-user design	11
Chapter III: the nonforgeable-identity theorem is false as stated	11
Chapter IV: the float plan confuses requester funds and provider collateral	11
Chapter V: attenuation must compare every edge, not every node to the root	12
Chapter VI: the game matrix and repeated-game bound are wrong	12
Chapter VII: cross-harbor resource substitution is a new grant	12
Cross-cutting: cryptography cannot prove reality	12
A better assurance argument	13
Exercise solution key	13
Chapter I - The Legible Swarm	13
I.1 - State of nature	13
I.2 - Consent and legible authority	14
I.3 - Digest-with-zoom	15
I.4 - Authority and completion	15
I.5 - Operator attention	15
I.6 - Read-poverty	16
I.7 - Directory and split ranker	17

I.8 - Tokens and compaction	17
I.9 - Canon mapping	18
Chapter II - The Single-Writer Kernel	18
II.3 - Seven organs	18
II.4 - One writer and schema evolution	19
II.5 - Durability by fault class	19
II.6 - Claims, ranges, and fairness	19
II.7 - Bus and markers	20
II.8 - Regimentation versus detection	20
II.9 - Continuity and checkpoints	21
II.11 - Consistency and runtime parity	21
II.12 - Cross-organ atomicity	21
II.13 - Threat boundary	21
Chapter III - From Spawn to Person	22
III.1 - Dependency chain	22
III.3 - Role, capability, permission	22
III.4 - Connectedness, continuity, and forks	23
III.5 - Three continuity organs	23
III.6 - Identity and whitewashing	23
III.7 - Local identity versus cross-operator attestation	24
III.8 - Reputation substrate	24
III.9 - Why public reputation is not a bandit	24
III.10 - Multi-dimensional grades and skill effects	25
III.11 - Rating the raters	25
III.12 - Tombstones and bounded memory	26
Chapter IV - The Harbor Economy	26
IV.2 - Market prerequisites	26
IV.3 - Three sides	26
IV.4 - Float plan and settlement	26
IV.5 - Rental and licensing on one escrow	27
IV.6 - Cross-operator identity	27
IV.7 - Conservation and valuation	28
IV.8 - Four-message transfer	28
IV.9 - Gossip and equivocation	28
IV.10 - Strategic graders	29
IV.12 - Cold start, cartels, resale	29
IV.13 - Gluing analogy	29
Chapter V - The Anchor Protocol	30
V.5.3/V.D.1 - Multi-hop attenuation	30
V.6.3 - Kani claims	30
V.7 - Security envelope	30
Chapter VI - The Bonded Commons	31
Correction to VI.7.4 before the printed exercises	31
VI.C E1 - Advisory claims	31
VI.C E2 - Reputation instead of bonds	31

VI.C E3 - Magic-link ProVerif model	31
VI.C E4 - Coverage-bound A5	31
VI.C E5 - Cartel detection	31
VI.C E6 - Capability attenuation	32
VI.C E7 - Cuckoo-filter pollution	32
VI.C E8 - Alternative insurance pricing	32
VI.C E9 - Gossip proof	32
Chapter VII - The Federated Harbor	32
VII.4 - Cross-harbor attenuation correction	33
VII.12.1 - Multi-principal correlation	33
VII.12.2 - Non-fungible trustless settlement	33
VII.12.3 - Cross-federation cartels	33
VII.12.4 - Cold-start admission	33
VII.12.5 - Equivocation propagation	34
VII synthesis check	34
Compact list of corrected premises	34
Local Harbor: the institutional runtime	34
Typed inter-agent messaging	34
Sole responsibility as a role lease	35
Buffered inputs, paging, search, and provenance-preserving compaction	36
The SLM sidecar as an untrusted projection service	37
The daemon as an effect hypervisor	37
Rename Vibe time and split the clocks	38
Critical Decision Method for skill extraction	38
Real checkpoints for arbitrary providers	39
Intra-agent resurrection and the big resume button	39
A GitHub review fleet with epistemic independence	40
Remote Project Harbor: Alice, Bob, and Carl	40
The coordination graph	41
Who decides what is a conflict	41
Lookout and Unspider as software-development prescience	42
Product and user experience are first-class	42
Sharing the durable agentic substrate	42
Federation boundary: CALM and invariant confluence	43
Sealed Harbor: Derek and Erin	43
The confidential work order	43
Bring the encrypted agent to the encrypted data	44
Two fences and no ambient authority	44
Coarse information-flow control	44
The release gate is the real security boundary	45
Provenance and receipts	45
Formal claim worth attempting	45

What cannot honestly be promised 46

Theoretical undergirding and honest bounds 46

 A formal work-unit transition system 47

 Experiments that matter 48

Build sequence 48

 Phase 0: repair the paper before adding machinery 48

 Phase 1: the one-agent receipt 48

 Phase 2: roles, messaging, and confined effects 49

 Phase 3: the review and foresight applications 49

 Phase 4: provider-portable resurrection 49

 Phase 5: hosted project institution 49

 Phase 6: Sealed Harbor 49

 Phase 7: economy and underwriting 49

Proposed new volume 49

Final provocation 50

References and theoretical anchors 50

Reader's note

This memorandum responds to the August 2026 collected volume as both manuscript and design program. It tries to do four things at once: say what is genuinely exciting, identify claims that do not survive close reading, solve the exercises rather than admire them, and develop the next architecture implied by the papers.

The short verdict is unusually positive and unusually sharp:

There is a real idea here, but it is not primarily an economy of artificial persons. It is an epistemic and effect-control system for accountable autonomous work. The manuscript keeps almost discovering that stronger thesis, then covering it with personhood, reputation, and market machinery.

The most valuable move now is to make the unit of theory a durable, evidence-bearing work unit owned by a principal. Agents, models, providers, and processes may change underneath it. The institution persists.

This reframing absorbs the best parts of the seven papers, makes provider-portable resurrection a first-class product, gives the remote Harbor a coherent ontology, and supplies a principled path to a mutually confidential Harbor for organizations that cannot expose either data or agent technology.

Executive verdict

What is alive

The manuscript sees several things earlier and more clearly than most agent frameworks:

- Oversight fails first as a reading problem, not merely a scheduling problem.
- A green status without inspectable evidence is an assertion, not knowledge.
- Durable responsibility requires state that outlives an individual model invocation.
- The human's attention is a scarce control resource and should be allocated by expected decision value.
- Authority, observation, recovery, adjudication, and compensation are different mechanisms.
- A hosted coordination system can become an institution rather than another orchestration library.

Those insights cohere. They point at **evidence-bearing autonomy**: work is safely delegable only when authority, effects, claims, evidence, recovery, and settlement have a common lineage.

What is obscuring it

The personhood and sovereign-economy story currently receives more formal and rhetorical weight than it has earned. It creates four distortions:

1. It assigns strategic agency to model processes when the economically meaningful actor is usually the human or organization that owns their utility, credentials, and liability.
2. It makes checkpointing appear necessary for reputation, when attributable witnessed outcomes need cryptographic identity and durable lineage, not psychological continuity.
3. It encourages philosophical analogies to carry claims that need transition systems, fault models, or experiments.

4. It pulls the manuscript toward a general market before the local product wedge is fully specified.

This does not mean deleting personhood. It means demoting it from foundation to optional policy.

Identity should answer: which liabilities, permissions, evidence, preferences, and commitments does a successor inherit? That is an inheritance rule, not a claim about a soul.

The thesis you are almost stating

Define a work unit as:

```
W = <principal, actor-build-lineage, intent, acceptance-policy,
    authority-envelope, expected-effects, claims, evidence-obligations,
    checkpoint, risk-guarantee, outcome>
```

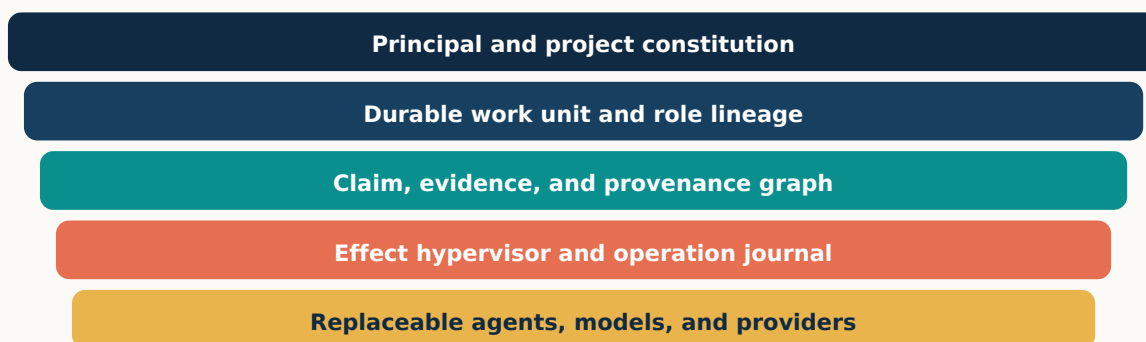
Its lifecycle is:

```
proposed -> authorized -> executing -> witnessed
          -> verified | contested | abandoned
```

Before execution, this object is a coordination permit. After execution, it is a work receipt. If a worker dies, changes provider, or is replaced, the work unit survives and its actor lineage grows.

The revised central claim can be stated without metaphysics:

Autonomous work becomes safely delegable when each consequential work unit is authorized before execution, mediated at its effect boundaries, recorded as evidence-bearing lineage, reviewable through provenance-preserving projections, recoverable across actor replacement, and settled against a durable principal.



Durability increases upward; replaceability increases downward

Product consequence

The wedge should work at one agent:

- a typed completion claim with evidence;
- a compact operator view whose statements zoom to sources;
- a visible authority and effect envelope;
- a task capsule that restores work under a minute;
- an honest receipt saying what was verified, what was merely observed, and what remains unknown.

That is already useful. A market, insurer, and federation can later price the same receipt. The strong commercial metaphor is not "Upwork for agent persons." It is closer to **Lloyd's for autonomous work**: standard work orders, declared risks, evidence, underwriters, adjudication, and guarantees.

What deserves lavish development

1. Claim discipline as the epistemic kernel

The volume's best rule is that state should move from asserted to observed to verified, with contested, superseded, and unknown remaining first-class. Push this much further.

A digest should not be a paragraph attached to an artifact link. It should be a projection of a claim-evidence graph. Use a claim object like:

```
c = <proposition, subject-version, author, derivation,
      evidence-refs, verifier, verdict, uncertainty, freshness>
```

The operator must be able to ask:

- Why is this claim present?
- Where did its inputs come from?
- Why is an expected claim absent?
- What would change the verdict?
- Which later claims depend on it?

Database why-, where-, and why-not provenance is the missing theoretical literature. It also resolves the "counter-surface" regress. The stable base is not another ranking interface. It is an append-only decision ledger plus queryable provenance. Every projection may be inspected against that base, and independent random audits can sample what the projection omitted.

The distinction matters: an artifact link is necessary but not sufficient. A test log can exist without warranting the proposition "the migration is safe." The graph must encode the derivation and the verifier that licenses that inference.

2. Attention as control, not dashboard aesthetics

The operator model is a genuine contribution, but the current Signal Detection Theory section needs repair. The useful decision quantity is not a generic risk score. It is the expected value of inspection per second:

```
priority(x) = [E loss(no inspection) - E loss(inspection)]
              / E operator-seconds
```

This automatically accounts for stakes, irreversibility, anomaly likelihood, verifier quality, available remediation, and the operator's comparative advantage. It also exposes a key limit: if inspection cannot change the decision, an alarming item can still have low inspection value.

The attention queue is a policy actuator. Treat its ranking decision as an auditable event with features, policy version, omitted alternatives, and counterfactual reasons. Do not recursively build a dashboard about the dashboard.

3. The principal above the actor

The principal layer is one of the manuscript's most important late discoveries. Promote it. The principal owns the objectives, externalities, credentials, economic exposure, and authority to create or replace agents. This repairs Sybil arguments, bond accounting, responsibility, and cross-harbor trust.

An agent may have an operational identity and a durable record. But a thousand agents minted by one principal must not create a thousand independent credit limits. Exposure, collusion analysis, and liability aggregate at the principal or principal-group level.

4. The institution that outlives the repo

The remote-harbor question reveals a larger product than shared Git coordination. Code is only one artifact projection. The durable substrate also contains goals, user needs, decisions, responsibility, agent builds, skills, connector descriptions, memories, consents, evidence, and outcomes.

That substrate can become more valuable than the repository because it explains why the repository exists, what future it is meant to realize, and which autonomous commitments are already in flight.

5. Mutually confidential computation

The Derek-and-Erin case is not an edge case. It may be the cleanest market reason for a remote Harbor. Derek has proprietary reality. Erin has proprietary cognition. The product is a two-key workroom in which neither side has to surrender its crown jewels, yet both receive an attributable, policy-bound outcome.

This is much stronger than generic "secure agent hosting." It combines a confidential clean room, an untrusted-agent sandbox, principal-owned declassification, evidence-bearing work receipts, and economic settlement.

What is not yet interesting

Several components are useful engineering, but they are not independent research contributions:

- A chat bus without typed institutional semantics.
- A claims or locks table that proves uniqueness only for exact keys.
- A vector-memory layer or summary cache without provenance and evaluation.
- Generic role labels without authority, obligations, fencing, and succession.
- JWT attenuation tutorials without a complete delegation and revocation model.
- Another scalar reputation estimator.
- Merkle roots presented as if they establish truth or completeness.
- A list of philosophical analogies standing in for a system model.

These become interesting only in combination: typed communication that creates durable obligations; roles whose effect authority is fenced; compaction whose omissions are auditable; receipts whose evidence can be underwritten; checkpoints that migrate work between providers; and confidential execution whose release policy is explicit.

The volume could lose roughly forty percent without losing a single core idea. Repeated reader maps, estimator catalogs, tutorial-level token material, ornamental theorem labels, and the recurring Hobbes/Locke/Parfit/Sen detours should be cut or moved to notes. Philosophers are excellent sources of questions. They should not be made to certify a systems guarantee.

Correctness audit: claims that must change

Chapter I: Signal Detection Theory points the wrong way

The diagram defines evidence of danger as increasing to the right and forces zoom above a criterion. Under that convention, moving the criterion left produces more zooms, fewer misses, and more false alarms. The prose says the opposite for a lax criterion. That is a directional error, not wording.

The objective also appears to count attention twice if C_{fa} already includes wasted inspection and a separate $a * E[\text{zooms}]$ term is added. Reputation r is named but not linked to either the signal distributions or the loss, so Exercise 5.1 is underdetermined as written.

Repair it by specifying:

```
z ~ p(z | danger, r, verifier-state)
inspect iff z >= beta
L(beta) = C_miss * P(miss | beta)
         + C_inspect * P(inspect | beta)
         + C_delay * E[delay | beta]
```

Then derive the likelihood-ratio threshold from costs and priors. If reputation is used, say whether it shifts the prior, the observation model, or the cost of a miss. Do not let it act as a mystical trust scalar.

Chapter I: roadmap-as-truth repeats the Scott failure

A roadmap is an authoritative statement of intent, priority, and dependency. It is not truth about the world. The repo and runtime expose artifacts and observations; the event ledger records what happened; claims describe what participants believe; outcomes test whether intentions worked.

Calling the roadmap truth turns learning into drift and incentives into map maintenance. Make it a versioned, amendable projection of accepted goals and commitments. A roadmap owner should reconcile and compile it, not govern reality by editing it.

Chapter II: exact-key uniqueness is not mutual exclusion

The mutual-exclusion theorem proves at most one live row for one exact normalized key. It does not prove exclusion for overlapping ranges or hierarchical resources. $[10,30]$ and $[20,40]$ have different keys. A whole-file claim and a symbol claim can also overlap.

You need either canonical conflict classes, an interval or hierarchy conflict predicate checked transactionally, or conservative coarse-grained claims. State the theorem over the predicate actually enforced.

Lease expiry also needs a fencing epoch. Without a downstream effect broker rejecting stale epochs, a paused holder can wake after expiry and act concurrently with its successor. The registry alone can prove one current owner in the table, not one real editor.

Chapter II: WAL auto-checkpoint does not bound elapsed loss time

SQLite's WAL auto-checkpoint setting is a page threshold, not a timer. It cannot bound δ in elapsed time. A quiet database may leave a commit uncheckpointed indefinitely, and a busy or reader-pinned database may be checkpoint-starved.

State durability by fault class and by write path. Use `synchronous=FULL` or an explicit durable commit protocol for money, authority changes, and non-repeatable external effects. A settlement ledger can pay the latency that an advisory marker does not need.

Chapter II: "reference monitor" overstates the same-user design

A reference monitor must be tamper resistant, always invoked, and small enough to verify. A same-user daemon with advisory hooks and bypassable Git, filesystem, process, and network paths is a coordinator, not a complete reference monitor.

Use explicit assurance modes:

1. Observed: effects may occur outside the daemon; evidence is best effort.
2. Coordinated: well-behaved clients use claims, roles, and commitments.
3. Brokered: credentials and selected effects pass through daemon-owned brokers.
4. Confined: the process has no ambient route around filesystem, network, tool, and credential brokers.
5. Attested: a remote party can verify the confined runtime and policy measurement.

The proposed effect hypervisor finally earns the reference-monitor language at levels 4 and 5, conditional on the isolation threat model.

Chapter III: the nonforgeable-identity theorem is false as stated

The theorem claims that a newcomer penalty prevents whitewashing. A counterexample is enough: let mature reputation be 10, sanction be 2, and newcomer reputation be 0. Keeping the sanctioned identity yields 8; resetting yields 0. Free identity does not make the sanction ineffective.

The stronger economic problem arises when a fresh identity receives access, credit, or opportunity comparable to a mature identity. State the condition in those terms. Aggregate exposure at the principal. Do not claim that checkpointing is a prerequisite for reputation; an identity key plus attributable witnessed outcomes is sufficient. Checkpointing improves service continuity and evidentiary reconstruction.

Chapter IV: the float plan confuses requester funds and provider collateral

The requester appears to fund both the bounty and a "worker bond," after which the bond is paid or slashed as if it were the worker's money. Separate:

- requester bounty escrow;
- provider performance bond;
- optional insurer reserve;
- fees;
- adjudicated transfer rules.

Returned collateral is not income. A partial settlement must state which evidence units were accepted and which party's posted asset moves. Amendments require both principals to sign a new plan version and atomically adjust escrows.

Chapter V: attenuation must compare every edge, not every node to the root

The main Anchor algorithm compares a delegated capability to the root in a way that admits `write -> read -> write`. The appendix finds the attack; the main algorithm and status table must be corrected. Each child must be no more permissive than its immediate parent, and the verifier must check the entire chain.

Also bind subjects to keys with proof of possession; specify ancestor revocation, unique token IDs, epoch and freshness rules, audience/resource semantics, and replay handling. Translating a resource across harbors is a new local authorization, not attenuation.

The Kani harnesses do not prove the prose property if they merely call a comparator on a few concrete vectors. A proof harness needs a relational assertion quantified over symbolic parent and child inputs. Two counterexamples or test vectors are not a universal proof.

Chapter VI: the game matrix and repeated-game bound are wrong

For the displayed payoffs:

	B:T	B:F
A:T	3,3	1,4
A:F	4,1	0,0

Against B:F, player A prefers T because $1 > 0$. F is not dominant, (F,F) is not a Nash equilibrium, and the pure equilibria are (F,T) and (T,F).

The repeated-game threshold is also wrong. At $\delta = 0.26$, deviating once from T,T to receive 4 and then playing T forever yields $4 + \delta/(1-\delta)$, while cooperation yields $3/(1-\delta)$. The deviation is profitable whenever $\delta < 1/3$.

More fundamentally, "true intent" is private and may be incoherent. Model observable effect claims, deposits, inspections, and outcomes. This is a hidden-type reporting and enforcement game, not an observed Prisoner's Dilemma.

Chapter VII: cross-harbor resource substitution is a new grant

A capability for a test database cannot attenuate into a capability for a staging database merely because the latter is described as analogous. The target harbor must apply a local semantic/resource mapping and mint a new authorization with an explicit dependency on the source grant. Revoking the source must invalidate the derivative, or the new grant must disclose that it is independent.

Cross-cutting: cryptography cannot prove reality

Signatures, hashes, Merkle inclusion, and append-only logs establish attribution, integrity relative to a commitment, and tamper evidence under stated assumptions. They do not establish:

- that every relevant event was logged;
- that the event description is true;
- that a test measures the intended property;
- that one party caused a business outcome;
- that an omitted channel does not exist.

Use typed evidence channels: deterministic verifier, provenance witness, protected meter, expert judgment, buyer preference, and delayed outcome. Every claim should name which channel warrants it and what that channel cannot show.

A better assurance argument

Do not ask one mechanism to do five jobs. Arrange defenses as a risk ladder:

1. Prevent the effect if a complete precondition can be checked.
2. Contain it within a narrow resource, credential, network, and time envelope.
3. Make it checkpointed or reversible where possible.
4. Observe and attribute it with evidence.
5. Adjudicate disputes under a declared oracle.
6. Compensate residual harm through bonds, guarantees, or insurance.

The underwriting layer should price what remains after the first five. Reputation is telemetry for the estimate, not the guarantee itself.



Residual risk moves right; proof burden changes at each rung

Exercise solution key

The answers below are deliberately concise. Where an exercise depends on a false premise in the manuscript, the solution repairs the premise instead of manufacturing the requested conclusion.

This is a compact answer key, not an endorsement of every premise in the questions. Where the manuscript's stated theorem, trace, or mechanism is false or too strong, the answer first gives the correction and then answers the intended question. Open problems receive a defensible design or proof obligation rather than a fictional closed-form solution.

Chapter I - The Legible Swarm

I.1 - State of nature

1.1 Check.

1. **Resource conflict:** two agents touch the same resource and clobber one another; answer it with conflict-aware claims plus actual exclusion at the relevant effect boundary.
2. **Illegibility:** work cannot be understood in bounded time; answer it with a digest whose every material claim zooms to independent evidence.
3. **Confident wrongness:** an agent asserts completion without checking; answer it with typed attestation and oracle-bound completion.

4. **Footguns:** a stale premise drives an irreversible act; answer it with reversibility/stakes gates, scoped authority, and pre-effect mediation.
5. **Illegible authority:** the coordinator rules without an inspectable reason; answer it with an append-only, named, zoomable decision record, including denials and omissions.

1.2 Check. Alice and Bob are both asked to improve `auth.ts`. Each reads the same base revision, makes a locally correct edit, and commits. Without a claim or merge serialization, Bob's whole-file rewrite erases Alice's change. No malice is required. A deadlock variant is two cooperative agents taking claims on files X and Y in opposite order and each waiting indefinitely for the other claim.

1.3 Trace. Deliberate construction chooses the authority rather than accepting whatever coalition happens to win, and it makes that authority legible to the operator. An emergent stable order may be an oligarchy: stable, but controlled by a few agents, unrevocable, and opaque. The local daemon also gives one fault domain, one policy root, and an unconditional operator exit.

1.4 Open. Randomize comparable real repository tasks to (a) an ungoverned fleet and (b) a fleet with only a transport, but no prescribed governance. Record all messages, effects, blocking decisions, and operator interventions. Pre-register two classes of observables:

- **Sovereign characteristics:** a publicly known rule, general applicability, explicit consent or adoption, stable appeal/override, reasons attached to authority acts, and equal audit access.
- **Oligarchy characteristics:** authority concentrated in a small coalition, private channels or rules, selective enforcement, no effective exit, and outcomes that cannot be reconstructed by outsiders.

Measure concentration of blocking/merging power, fraction of authority acts with reconstructable reasons, rule consistency across agents, override success, artifact-linked claim accuracy, and operator reconstruction accuracy. Stability alone does not separate the hypotheses; publicly inspectable and revocable rule-governed authority does.

1.2 - Consent and legible authority

2.1 Check. Tacit consent is inferred from remaining in or using a system. Express consent is a discrete act over named terms. Definition 1.2.1 is express because the operator affirmatively issues a grant with scope, stakes ceiling, reversibility floor, TTL, and revocation semantics.

2.2 Check. If an autonomy level can suppress the override, it can make its own authority irrevocable: keep executing, hide the stop channel, or race destructive effects after consent is withdrawn. The override must be out-of-band, higher priority than agent work, and coupled to effect revocation, not merely a UI flag.

2.3 Trace. The operator signs grant `g`; the daemon validates and records it; an eligible low-stakes action cites `g`; the daemon mediates the effect; the receipt records action, reason, artifact, and grant version; expiry or revocation removes `g` from the active set and fences later effects. The legible-sovereign rule binds at every authority transition, especially issue/revoke and admit/deny. The explicit witness record is where reconstructability is discharged.

2.4 Open; also 9.4. Use one complete append-only **decision ledger**, not a second ranked dashboard. For every candidate item, store candidate ID, source/evidence pointers, policy and model version, input features, score components, threshold/lane, surfaced-or-suppressed verdict, reason codes, expiry, and a counterfactual such as "would surface if anomaly were 0.08 higher." The operator can query suppressed since cursor, sort by any raw field, and sample items uniformly rather than through the production ranker. Decisions are immutable; their current relevance expires by signal-specific TTL and is recomputed from raw state. Run independent random omission audits and expose raw event-log access.

The regress terminates at completeness plus direct sampling: the base ledger records every candidate and every ranking decision, and the counter-surface is a deterministic query over it. Another learned ranker would recreate the regress. The remaining trust assumption is the complete event-capture boundary; that assumption should be tested and externally anchored, not hidden behind another summary.

I.3 - Digest-with-zoom

3.1 Check.

- Port number: flatten as structured administrative state.
- Failed-test stderr: preserve verbatim or by a content-addressed pointer.
- "The auth refactor felt risky so I split it": preserve verbatim and label it self-narration, not fact.
- Commitment status enum: flatten.
- 600-line diff: do not paraphrase away; retain the immutable diff or a verified pointer and summarize only administrative metadata.

3.2 Check. Two models may share training biases, prompts, sources, and failure modes. Agreement over the same narration is correlated testimony, not independent evidence. A second check must derive from an independent channel such as repository state, a protected test runner, or a separately controlled reviewer.

3.3 Trace. A typical CI dashboard often fails as follows: deploy is irreversible but the green tile does not force a log/diff view; administrative state and diagnostic detail are flattened together; the summary is generated from job narration rather than independently from receipts; and a skipped/cancelled check has no named authority reason. It therefore violates Definition I.3.1 wherever a material green claim has no total zoom path to the exact tree, suite, runner, and output that produced it.

3.4 Open. L2 can compare structured narration claims against independently derived artifacts: claimed changed files versus the tree diff, "tests passed" versus a protected runner receipt, "no deletion" versus Git objects, and claimed command results versus brokered tool logs. Emit a contradiction event on mismatch. This catches deterministic report/artifact divergence, deleted tests, fabricated execution, and omitted changed paths. It cannot establish semantic correctness, detect a poisoned acceptance suite, observe effects that bypass logging, or distinguish two artifacts that are syntactically consistent but jointly wrong. Those require effect confinement, independent evaluation, or later outcome evidence.

I.4 - Authority and completion

4.1 Check. Verifiable zoom says narration is not ground truth. A reviewer with no independent evidence would merely add another narration. Adversarial review follows because a consequential claim needs an evidence channel controlled independently of the actor whose work is judged.

4.2 Check. Property I.4.1 should be read as: prevent a prohibited state when every decisive effect can be intercepted before it occurs; otherwise detect, contain, and compensate without pretending prevention. A uniqueness constraint can regiment "one holder per exact key." A post-commit monitor can detect an out-of-scope edit and trigger rollback/salvage, but the bad prefix already happened.

4.3 Trace. For "the migration applies cleanly," the verifier is an isolated migration run against a declared base schema/database, with exact migration hash, runner image, exit status, and resulting-schema check. *done* and any merge/settlement are gated on that receipt. "The API feels ergonomic" has no mechanical truth predicate. It must go to a declared human/expert or plural judge with preserved evidence and uncertainty; absent that authority, completion remains provisional or fails loudly.

4.4 Open. Honest fallback ladder: (1) use a deterministic verifier where the property is decidable; (2) require an independent bounded evaluator or human for a declared subjective criterion; (3) record a provisional result, confidence, appeal window, and residual risk; (4) return unresolved/fail-loud when no authorized evaluator can support the claim. "I cannot verify this" is correct when the available evidence cannot distinguish success from failure under the contract. It is a cop-out only when an affordable, authorized verifier was specified and simply not run.

I.5 - Operator attention

Correction to Figure I.6 and Eq. I.1. With evidence of danger increasing to the right and "zoom if evidence exceeds beta," moving beta left is more stringent: it causes more zooms, fewer misses, and more false alarms. The figure's prose calling that criterion "lax" reverses the direction. Also, if c_{fa} already means wasted zoom attention, adding a $E[\text{zooms}]$ double-counts that cost; define c_{fa} to exclude attention or remove the extra term.

5.1 Check. As $C_{miss}/C_{fa} \rightarrow \text{infinity}$, the optimal criterion moves toward forced review wherever a miss remains possible; the forced-zoom probability approaches one on the ambiguous region. If reputation $r \rightarrow 1$, nothing follows from Eq. I.1 alone because r is absent. Zoom should decline only under a calibrated model in which high r shifts the class-conditional danger probability downward for this agent, action class, and current version. High reputation must not defeat hard stakes or irreversibility gates.

5.2 Check. An operator may be correctly confident in one agent's formatting/test-only changes and correctly skeptical of its schema migrations. A global trust knob averages those histories, forcing needless review of safe formatting while under-reviewing migrations. Calibration belongs to (principal, agent/build, action class, environment), with uncertainty and freshness.

5.3 Trace. Campbell pressure appears when canaries are selected (recognizable templates), exposed (agents/operators learn signatures), scored (catch rate substitutes for real defect discovery), and acted on (friction creates incentive to optimize the metric). Time-on-task and breaks also confound the score. Reduce distortion with a blinded, continuously refreshed holdout drawn from real failure transformations; separate the party generating canaries from the operator; correct for exposure time and base rate; and use catch rate as one diagnostic, never the sole gate.

5.4 Open. Model each digest policy d by (token cost $T(d)$, human time $H(d)$, expected decision loss $L(d)$) and compare nondominated policies. A Pareto frontier exists for a finite policy set, but there is no universal inverse relation: removing redundant context can lower both token cost and operator time; beyond that free compression frontier, preserving evidence often trades tokens for review effort or lower error. Optimize under a loss or safety constraint, for example $\min \alpha * T + \beta * H + L$ subject to zero omission of hard obligations. Report the frontier empirically; do not claim one universal digest optimum.

5.5 Open. Randomize operators or time blocks among manual-tax, automation-only, projection/freeze-probe, and adaptive-tax conditions. Use matched repositories and fixed surprise events. Probe without warning: next-three-effect prediction, takeover completion time, false-belief rate, Brier score, post-interruption recovery, defect catch, NASA-TLX, and retention weeks later. Measure disablement/avoidance as an outcome. A useful tax improves later projection/takeover after controlling for interruption cost; mere annoyance raises workload and disablement without those gains.

1.6 - Read-poverty

6.1 Check. Writes can be serialized through one daemon, rejected by uniqueness constraints, and reconciled through version control. Read demand grows with agents, artifacts, histories, and possible collaborators; without indices every query becomes an $O(n)$ scan and the operator is the fixed-rate server. Thus discovery and relevance, not raw write throughput, become the scale bottleneck first.

6.2 Check. A directory pays at write time to normalize metadata and update indices, typically $O(\log n)$ per indexed insert (or expected $O(1)$ for a hash index) plus storage. It buys selective reads in $O(\log n + k)$ for k results rather than $O(n)$ roster scanning, along with a queryable provenance trail.

6.3 Trace. At $n=1$, briefing, honest self-attestation, footgun gating, completion receipts, and resurrection handoff already reduce uncertainty. None asks "which collaborator fits?"; they project or verify one actor's state, so they are not discovery ranking.

6.4 Trace. The bound is $\log_2 C(N,k) - \log_2 C(m,k)$. At $k=0$ there is only one critical subset and the floor is zero. For small positive k , the cost is roughly $k \log_2(N/m)$ when $k \ll m, N$. At $k=N/2$, exact identification ($m=k$) costs about N bits, while opening almost everything drives the floor down and $m=N$ makes it zero. The danger fraction alone is not enough; the allowed review budget m matters. Mostly safe work is cheap only when the system can reliably identify that sparsity.

6.5 Open. The theorem is conditional on an encoder that knows the true critical set. If an adversary controls apparent criticality and can make a critical artifact observationally indistinguishable from inert artifacts, any policy that opens fewer than all N can be forced to miss one. Zero miss then requires inspecting all N , a trusted independent signal that restricts the adversary, or a randomized guarantee stated probabilistically. The combinatorial bound still applies after a true set is fixed; it does not solve adversarial set identification.

1.7 - Directory and split ranker

7.1 Check. No monotone scalar transform can identify collaborator fit with operator regret on a domain containing high-fit/low-risk and low-fit/high-risk items. Discovery minimizes routing loss or maximizes capability fit; the attention head minimizes expected loss from not inspecting, ideally value of inspection / attention second, not merely raw stakes times anomaly.

7.2 Check. The attention queue allocates a scarce human interrupt channel, so agents benefit directly from inflating its score and buried items can cause irreversible loss. Discovery gaming usually yields a bad routing recommendation and can be corrected before effect. Both Goodhart, but the queue has stronger incentives and consequences.

7.3 Trace. Without expiry, the dead holder's row remains authoritative and the live agent is denied forever. With a lease, heartbeat-based expiry, and a new fencing epoch, the stale lease becomes ineligible; a lazy sweep on the next acquire removes it; the live agent receives a higher epoch and proceeds. Merely decaying a display score is insufficient: ownership must actually expire, and every effect boundary must reject the stale epoch.

7.4 Trace. The escalation cites evidence and posts a small stake or reputation exposure. The operator's dismissal is itself reasoned and appealable; only a confirmed low-value/false escalation debits the signaler. Repeated unsupported alarms raise future cost or lower priority, while verified hazards return the stake and improve calibration. Never debit solely because an operator clicked dismiss: that would silence minority warnings and make authority error self-validating.

7.5 Open. Accept/reject is useful implicit relevance feedback but is confounded by position, exposure, workload, and incumbent policy. Learn with logged propensities, small randomized exploration, counterfactual evaluation, and a held-out labeled sample; preserve refusals and downstream task outcomes. If acceptance directly trains what is shown, the loop can entrench its own choices, so it cannot be the sole ground truth.

7.6 Open. Use hard expiry for leases/presence; state transition or supersession for commitments; summary plus source provenance for decisions/rationale; immutable storage for evidence; and eviction plus a content-addressed pointer for reconstructable diffs/logs. Hints may decay continuously. Never decay a still-open obligation or recursively summarize prior summaries.

1.8 - Tokens and compaction

8.1 Check. The operator digest and the context compaction must be the same provenance-preserving projection from durable sources. If a separate model summarizes the already-compacted agent story, cost control and truth control diverge and recursive-summary error compounds.

8.2 Check. Context past the model's effective window costs tokens while degrading retrieval and reasoning. Removing redundant/inert material and edge-placing hard facts can therefore reduce spend and improve accuracy simultaneously, provided obligations and source pointers survive.

8.3 Trace.

- **Briefing:** reader is an arriving agent; drops old conversational detail; keeps current roles, work, constraints, decisions, and source links.

- **Attention queue:** reader is agent/operator now; drops low-value candidates from the immediate view; keeps highest value-of-inspection items plus the suppression ledger.
- **Episodic memory:** reader is a future incarnation; drops transient chatter; keeps typed incidents, decisions, outcomes, and retrieval features.
- **Pheromone:** reader is the swarm; continuously drops stale coordination weight; keeps fresh shared traces.
- **Resurrection handoff:** reader is a successor; drops nonessential transcript detail; keeps the task capsule, artifacts, obligations, pending operations, and provenance.

8.4 Trace. Context rot -> compact earlier to an effective-window budget. Lost-in-the-middle -> shorten and edge-place mandatory facts. Recursive-summary collapse -> re-derive every compaction from raw durable artifacts. Over-flattening -> require a total zoom path from every material claim. The third cure prevents model noise from becoming the next round's source of truth.

8.5 Open. "Replay from digest alone" is neither sound nor gaming-resistant: a digest can overfit the smoke task or omit facts the sampled successor never needed. Give a successor the digest plus authorized content-addressed source links, then measure next-step success, false beliefs, obligation recall, time, and tokens on hidden continuation tasks. Add a cheap structural invariant: every open claim, decision, commitment, risk, and pending external effect has exactly one valid digest entry or pointer. Run expensive replay only on a random sample.

8.6 Open. Budget allocation over a precedence DAG is a knapsack/resource-allocation problem and is NP-hard in general. First reserve a minimum viable budget for each mandatory node and any predecessor needed to make its output exist. Allocate the remainder by estimated marginal outcome value per token while respecting precedence; recompute after each result. For a monotone submodular value function under a cardinality budget, greedy earns the standard $(1 - 1/e)$ approximation; for arbitrary complementarities there is no such bound. Dynamic programming is pseudo-polynomial for small integer budgets or trees. A reviewer's tokens have near-zero value until the worker artifact it reviews exists.

I.9 - Canon mapping

9.1 Check. Agents are the multitude bound by the common coordination protocol; the daemon is the sovereign-as-actor that applies the rules; the operator is the author/principal who originates and can withdraw authority. Calling the operator the sovereign collapses author and executing authority and makes the supposed covenant among agents incoherent.

9.2 Check. Voice is honest only if refusal has consequence for the authority. The unconditional override supplies exit: the operator can revoke the grant when explanations or appeals fail, rather than being forced to keep negotiating inside a system they cannot stop.

9.3 Trace. The grant is discrete, explicitly issued, scoped, time-bounded, and revocable/audited. "Started using the tool" has none of those four properties.

9.4 Open. Same answer as 2.4: complete decision capture, deterministic raw queries, independent sampling, and an externally anchored event boundary terminate the regress. Do not rank the ranker's omissions with the same ranker.

Chapter II - The Single-Writer Kernel

II.3 - Seven organs

Check. In-memory operation preserves transient versions of resource exclusion, identity/presence, communication, obligation checks, policy checks, and self-attestation while the process lives. The substrate still serializes. Durable continuity, crash recovery, historical audit, and outcome evidence become vacuous after process death; durable identity also collapses to an ephemeral handle. The organs remain conceptually meaningful, but their cross-crash guarantees disappear.

Trace. Example: the communication organ's home table is `messages`. It owes the layer above an ordered, cursor-addressable, durable envelope with explicit at-least-once semantics. Equivalent valid choices include `claims` for the resource organ (one compatible holder per conflict domain) and `commitments` for obligation (no done without a current structured oracle receipt).

Open. A useful nine-way cut is: storage/durability, transaction serialization, identity/presence, resources/leases, message transport, stigmergy/projections, policy/capability, commitments/outcomes, and attestation/recovery. It separates proof obligations that currently leak across organs, especially transport semantics from message semantics and policy prevention from post-commit detection. It hides the simplifying fact that all local state shares one transaction boundary, so retain a cross-cutting "one ledger/effect mediator" invariant.

II.4 - One writer and schema evolution

Check. Module A renames `users.name` to `display_name`; module B, loaded later under old code, sees no name and lazily adds it back. Writes now split between two columns according to module version/load order, and neither backfill is authoritative. Any destructive migration has this problem.

Trace. CLI -> authenticated Unix-socket request -> daemon request queue -> sole read/write connection -> transaction -> WAL commit. Routing enforces the discipline at the client/API boundary and at the daemon's sole-connection boundary; SQLite's file lock is one backstop at write time. The primary story is routing, not a database rule that forbids other connections.

Open (OP-7). Expand/backfill/dual-read-or-write/contract can keep mixed versions safe temporarily, but the contract step and dependency order are global facts. Store a migration ledger with schema epoch, module compatibility range, dependencies, and backfill watermark. Idempotent self-init can remain for additive tables/columns; safe rename/backfill/down-migration cannot preserve unrestricted "any module, any order." A version or equivalent ordered capability ledger is unavoidable.

II.5 - Durability by fault class

Check. (a) forced daemon kill: I1a-survivable; (b) a genuinely clean OS reboot: normally flushed during orderly shutdown, so it is not the I1b fault case; an abrupt reset during reboot is I1b-exposed; (c) power cord: I1b-exposed; (d) unhandled handler exception/process crash: I1a-survivable after commit. None says an uncommitted transaction survives.

Trace - correction. A commit is power-loss exposed from t_0 until the WAL containing it is synced. A page-count auto-checkpoint threshold bounds accumulated WAL frames, not elapsed time Δ . At a low or zero write rate the threshold may not fire for arbitrarily long, so there is no wall-clock bound. With an independently guaranteed lower write rate $w_{\min}$ pages/second and threshold P , one could derive approximately $\Delta \leq P/w_{\min}$; the manuscript states no such assumption.

Open (OP-10). Expose `durability = PROCESS_CRASH | POWER_LOSS` on a transaction, not an adjective in prose. Route `POWER_LOSS` operations through a connection/database configured `synchronous=FULL` before the transaction and verify commit before returning; a separate `FULL-sync` financial ledger is the cleanest isolation if SQLite settings are connection-scoped. Keep ordinary claims on `NORMAL`. Hard-reset fault tests must validate the stronger path. A forced checkpoint can bound recovery/log size, but `FULL` commit sync is the relevant per-write durability primitive.

II.6 - Claims, ranges, and fairness

Correction to Theorem II.6.1. A uniqueness constraint proves at most one row per exact key. It does not exclude overlapping line intervals, a whole-file claim versus a symbol claim, path ancestry (`dir/` versus `dir/file`), or semantically overlapping resources. Those require canonical conflict domains or a transactional overlap predicate/index. The theorem must be scoped accordingly.

Check. Releasing with no matching (`key`, `holder`) is a successful no-op so retries are idempotent (I3). It must not delete another holder's row. Raising on an already-completed release would make

timeout/retry clients unable to distinguish success from failure and would break at-least-once handling.

Trace. The daemon serializes the requests. The first `INSERT(key, holder)` commits. The second hits the unique-key violation, selects the extant row, and returns `DENIED(holder=first)`. For range/hierarchy claims this trace is safe only after both requests map to the same canonical conflict key or the overlap test is done in one immediate transaction.

Open (OP-1). Add a durable FIFO ticket queue per conflict domain, maximum lease L , monotonically increasing fencing epoch, and grant-to-head on release, acquire, or lazy expiry. No background scheduler is required for safety, but liveness requires a live caller/tick to trigger transitions. Under daemon availability, eventual requests, and non-renewable bounded leases, a requester with q predecessors waits at most roughly $(q+1)L$ plus processing delay. Without those assumptions no bounded wait exists. Every external effect must validate the fencing epoch or an expired holder can still act.

II.7 - Bus and markers

Check. If the evaporation tick stalls for an hour, tick-only storage still reports yesterday's marker weight. Read-time evaluation computes $w * r^{\Delta t}$ from the persisted timestamp and therefore never presents the stale stored number as current.

Trace. Transport delivers one request twice. The consumer applies one semantic state transition; the duplicate is silently recognized as transport nondeterminism, while diagnostics may count the redelivery. An envelope idempotency key plus a unique (consumer, key) receipt lets the consumer atomically store "effect applied" with the effect, making retry safety explicit. "Exactly once" is still not guaranteed for an unbrokered external effect.

Open (OP-8). For each marker kind, log creation, reads, actions influenced, supersession, and the time at which operators label it stale. Fit a survival/hazard curve and choose the half-life minimizing $C_{\text{stale}} * \text{false-live} + C_{\text{lost}} * \text{false-expired}$, validated on held-out projects. Claims/presence should decay far faster than decisions or learned skills. Publish confidence and re-estimate under drift.

II.8 - Regimentation versus detection

Check. Note-count monotonicity can be regimented by append-only storage or a trigger rejecting delete/regressive update. Capability attenuation and lock-owner release can be prechecked only if authenticated identity, the capability order, and every relevant effect path pass through the mediator. Duplicate keys and boot admission are already pre-effect. Heartbeat freshness cannot be rejected at the heartbeat insert: staleness is an absence observed only after time passes, and under asynchrony a slow actor is indistinguishable from a failed one. Out-of-band filesystem/network effects remain only detectable until mediated.

Trace. `close(done, oracle_ref="")` reaches line 4 of Algorithm II.2, returns `REFUSED`, leaves the commitment open, and records the denied authority act. It never reaches classification, current-state verification, or the `DONE` transition.

Open (OP-2). A safety property is regimentable iff every event that can complete one of its finite bad prefixes crosses a trusted mediator before its real-world effect and membership in the allowed next-event set is decidable there. Then suppressing the event enforces the property. If the decisive event is observed only after commit, occurs outside the mediator, depends on future absence, or needs an undecidable semantic judgment, the mechanism can enforce only detection/containment/recovery properties.

Open (OP-5). Add a `PROTECTED_RUN_RECEIPT` oracle containing tree hash, suite hash controlled by the principal, runner/image hash, command, timestamp/nonce, exit code, and signed result. Closure verifies all bindings and freshness. This resists free-text and stale-result manipulation; it is not universally Goodhart-proof because the suite may be incomplete or poisoned. The honest theorem is syntactic provenance and non-replay, not semantic correctness.

II.9 - Continuity and checkpoints

Check. Notes preserve explicit goals, rationale, observations, and artifact pointers. They do not preserve unflushed edits, process memory, open handles, provider KV cache/hidden state, or a latent next action the model never externalized. A successor may understand the story yet be unable to resume the exact computation.

Trace. The append commits. A later delete/regressive count also commits and enters the operation log. Only then does the subscriber observe the count drop and raise/compensate. The violation is already durable at the second commit; the monitor proves detection, not prevention.

Open (OP-4). Use a content-addressed task capsule at a safe tool/message boundary: base tree and worktree blobs, environment/tool manifests, open claims with epochs, commitments/acceptance criteria, exact tool I/O, pending external operations and idempotency keys, decisions/hypotheses, transcript or provenance-preserving compaction, and external references. Flush the ledger and quiesce mediated effects before sealing it. This restores observable task state and evidence. It cannot recover hidden activations, unexposed reasoning, provider-internal caches, or unrecorded intent; arbitrary-provider recovery is semantic continuation, not bit-identical resurrection.

II.11 - Consistency and runtime parity

Check. The four assumptions are: one daemon/connection handles all claim/lock reads and writes; no read_uncommitted; respond only after commit; and no out-of-band writer. A write-capable external SQLite connection directly violates the first and fourth (the out-of-band-writer assumption is the most literal first failure). A read-only connection need not break write linearizability but may observe under different timing/API semantics.

Trace. A invokes claim(k) and commits at c1. B and C overlap; B's denied read commits/returns at c2, A releases at c3, and C's acquire commits at c4. One valid order is A-acquire < B-denied < A-release < C-acquire. If B and C overlap around the release, either may linearize first as long as each result matches the chosen state and non-overlapping real-time order is preserved.

Open (OP-3) - correction. A finite differential suite cannot "make I11 a theorem." It supplies bounded evidence. Generate identical state-machine traces against both bindings: schema init/migrations, acquire/reclaim/release/expiry, overlapping calls, transactions and rollback, busy/lock behavior, encodings, PRAGMAs, crash/reopen, and fault injection. Compare returned values, errors, serialized rows, schema, and recovery state after every step, including the deployed binary in CI. A theorem requires a formal refinement/equivalence proof or the same verified binding; tests should be labeled conformance evidence.

II.12 - Cross-organ atomicity

Check. The port remains claimed and a live/ghost session row exists, but there is no commitment saying what work owns them. Other agents see a busy port and session presence they cannot reconcile; the failed starter may retry into its own leftovers.

Open (OP-11). Expose one begin_work(...) endpoint that starts BEGIN IMMEDIATE, validates all inputs, inserts claim/session/commitment/outbox rows, and commits once; on any error it rolls back all. A Saga would release the claim and mark/delete the session if the commitment insert failed, but those compensations can themselves fail or crash between steps. Since all three rows are in one SQLite file, the local transaction is both simpler and stronger.

II.13 - Threat boundary

Check. A local hash chain can expose edits by a later same-user database tamperer or by a remote synchronizer that cannot rewrite an externally anchored root. The same-user process is excluded locally; the adversary becomes meaningful in the cross-machine/economy layer. Without an independent anchor or verifier, the writer can rewrite the log and recompute the chain.

Trace. The release path compares the supplied actor ID to the lock owner. If a legacy endpoint accepts a self-asserted ID, the attacker submits the victim's string; the equality check passes and the

owner-scoped delete executes. The lock rule is correct relative to its input, but I12 failed to authenticate that input. Complete credential/peer binding must precede every security-relevant write.

Open (OP-9). The minimum useful wall is a daemon under a separate UID (or VM), daemon-owned DB/key, Unix-socket ACL plus kernel peer credentials, and brokered Git/filesystem/network effects. An OS keychain protects a key at rest but not arbitrary signing requests from an equally authorized same-user process. "Sealed memory" does not stop same-UID ptrace/injection without an OS isolation policy. External root anchoring detects later log rewrite but does not prevent it. No TPM is required for strong local separation; separate privilege and complete mediation are.

Chapter III - From Spawn to Person

III.1 - Dependency chain

Check (1). A spawn is a temporary process filling a role; a person is a role-holder whose accountable continuity survives replacement of that process.

Trace (2). Read right-to-left:

- No reputation -> a buyer cannot price differentiated trust, so the alleged tradeable asset is adverse-selection noise.
- No durable accountable person/outcome identity -> reputation attaches to a disposable incarnation and can be reset or duplicated.
- No continuity -> successive role-holders cannot be shown to belong to one lineage; history does not follow replacement.
- No memory/checkpoint/evidence -> a successor lacks both resumable state and a witnessed chain from old work to new work.

The chain is not literally linear: non-forgable identity and witnessed outcomes are parallel prerequisites for accountable continuity; checkpointing is valuable for resumption but is not logically necessary for every reputation system.

Open (3). The human operator is a moral/legal person but should be represented economically as the **principal** above agent identities. If one principal can mint unlimited apparently independent agent-persons, it can whitewash sanctions, forge quorums, reuse aggregate collateral, and farm reputation through self-trade. Bind every agent lineage to a principal credential, cap principal-wide exposure and newcomer credit, and disclose shared control. The actor may change; the economic liability root must not.

III.3 - Role, capability, permission

Check (1). Capable-and-permitted: an agent has a test-runner tool and authority to run the suite.

Capable-but-forbidden: the process can invoke `git push --force`, but policy forbids it.

Permitted-but-incapable: the role is authorized to deploy, but the incumbent has no deploy credential/network route.

Trace (2). Reputation attaches to the person's continuity witness and witnessed outcomes, not to the abstract role's O, C, or A. A role is fillable and history-free. Otherwise a fresh spawn could inherit the prior holder's credit merely by taking the same job title.

Open (3). Keep a role template (O_{required} , A_{role}) separate from an incumbent capability set C_{subject} . At assignment require each current obligation to satisfy $O_{\text{active}} \subseteq A_{\text{role}} \cap C_{\text{subject}}$. Keep $C_{\text{subject}} - A_{\text{role}}$ as capable-but-forbidden and $A_{\text{role}} - C_{\text{subject}}$ as authorized-but-unavailable, which triggers provisioning or reassignment. Never equate C with A.

III.4 - Connectedness, continuity, and forks

Check (1). Connectedness is a direct memory/intention link between two incarnations; continuity is the transitive, provenance-bearing chain across arbitrarily many links. A successor may read its predecessor's handoff and be psychologically connected while using a new outcome-ledger key, so the accountable chain is broken: connected, not economically continuous.

Trace (2). In the manuscript's analogy, the body-lease is the replaceable live substance/process; the actor-soul is the durable identity/mailbox/history that carries consciousness-like continuity. Respawn replaces the body-lease and should retain the actor-soul only under a valid succession event.

Open (3). Represent a fork as a lineage DAG, never as two copies of one linear identity. Both children may display the ancestor's evidence with an explicit inherited/discounted prior, but neither duplicates the ancestor's outstanding authority, collateral, or spendable reputation. Allocate a fresh branch ID, split or revoke exclusive capabilities, and aggregate risk at the common principal/ancestor. Copying full reputation to both invites credit duplication, quorum multiplication, and double-spending of grants; giving it to only the first child creates a fork-race attack.

III.5 - Three continuity organs

Check (1). The checkpoint organ is the specifically weak one: it forwards notes rather than execution state. The outcome organ is also only partial in the implementation: commitments and oracle closure exist, but neutral graded outcomes and reputation binding do not.

Trace (2). Memory retains the session, notes, and explicit decisions. The current checkpoint may retain a handoff and perhaps a committed diff, but not working process state, tool handles, pending effects, or hidden provider state. The outcome ledger retains the commit only if it is bound to a work unit and independently witnessed; a naked Git commit is an artifact, not proof of accepted delivery. A strong checkpoint would also preserve the worktree, environment, open claims/epochs, acceptance state, tool receipts, and idempotency journal.

Open (4). Useful death taxonomy:

1. **Record survival:** logs/notes only; reconstruction is manual.
2. **Artifact survival:** content-addressed worktree and outputs survive; computation is rerun.
3. **Semantic task capsule:** artifacts plus obligations, decisions, transcript/projection, tools, and pending-effect journal; a new provider can continue observably.
4. **Execution snapshot:** process memory/runtime state survives under a controlled runtime; rarely portable.
5. **Latent state:** unexposed activations, KV cache, or "intent" dies; it is fundamentally unrecoverable from an arbitrary provider.

Checkpoint guarantees should state which tier they provide.

III.6 - Identity and whitewashing

Correction to Theorem III.6.1. Free fresh identities do not by themselves imply that every sanction disappears. Counterexample: a mature identity has score 10, is sanctioned to 8, and a fresh identity starts at 0; with free minting the actor's best accessible score remains 8, a real reduction from 10. The proof's equality $\max(r(i) - \Delta, r_0) = r_0$ holds only when $r(i) - \Delta < r_0$, and the conclusion also needs fresh identities to receive equivalent economic access. The valid theorem is conditional: if an actor can costlessly obtain a fresh identity whose accessible utility is at least its sanctioned utility (or can replicate identities to defeat a quorum), identity-local sanctions/weights are evadable.

Check (1). Correctly scoped, Property III.6.1 says reputation has economic force only to the extent that the accountable principal cannot cheaply discard or multiply the liability-bearing identity. Whitewashing escapes a negative record when newcomer access is better; a Sybil attack multiplies votes/credit when the mechanism counts identities. Non-forgability is necessary for those mechanisms under those access rules, but it is not sufficient and the printed universal theorem is false.

Trace (2). Full work with a reduced economic ceiling lets an honest newcomer demonstrate competence without being denied livelihood, while limiting the maximum one-shot gain from abandoning a mature sanctioned identity. "Reduced work" slows evidence accumulation and taxes all honest entrants; "reduced exposure" targets the externality. The ceiling schedule must be chosen so the maturation opportunity cost exceeds plausible evasion gain.

Open (3). Principal binding is the load-bearing defense: aggregate reputation, exposure, counterparties, and sanctions across the 1,000 agents, so self-trades add little or no independent evidence. A listing fee raises attack cost but only deters while total fees exceed the minted value. Sampled adversarial re-audit detects fabricated work, with false-positive burden approximately the sampling rate times the honest high-volume population and adjudication error. Add sharply diminishing credit per principal pair, independent payer stake, and principal-wide reserve limits.

III.7 - Local identity versus cross-operator attestation

Check (1). Local non-forgable identity proves that, inside one trusted daemon, an actor cannot freely impersonate or re-pick its daemon-issued handle. Cross-operator attestation must additionally bind a key to a durable principal across a hostile operator boundary, establish proof of possession/freshness, aggregate related agents, and support revocation and audit.

Trace (2). "Alice and Bob settle across a boundary neither controls" secretly assumes the cross-operator stone. With only local identity, Bob learns at most "Alice's daemon asserts this actor/key." A hostile Alice can mint arbitrary local actors, rewrite local history, or misstate principal separation.

Open (3). A witness log proves publication order and exposes equivocation; it does not prove who controls a key or that two keys are distinct principals. Close the gap with a scarce external root: hardware/remote attestation, organizational/KYC issuer, bonded sponsor chain, or a combination. A transparency log makes bindings and rotations auditable but not true; a sponsor creates economic accountability but can cartelize; a shared identity issuer centralizes and leaks privacy; hardware roots attest devices/code, not unique humans. State the chosen trust and Sybil assumptions.

III.8 - Reputation substrate

Check (1). With a complete, mostly stable paired-comparison history, batch Bradley-Terry uses all observations and is more appropriate than order-sensitive streaming Elo. If abilities drift, use a time-varying/dynamic model rather than pretending full-history stationarity.

Trace (2). A binary protected acceptance test can update a beta-Bernoulli reliability estimate (or a calibrated per-task success model). Human aesthetic preference is naturally pairwise and may use Bradley-Terry/TrueSkill with judge effects. EigenTrust expects a who-trusts-whom graph and therefore fits neither raw task-outcome signal directly.

Open (3). If identities are self-chosen, an exact Bayesian estimator simply produces precise scores for disposable names. If the agent authors the "oracle," a sophisticated estimator precisely learns manipulated labels. Better statistics cannot repair missing provenance, adversarial labels, or liability binding; secure the event-generation substrate first.

III.9 - Why public reputation is not a bandit

Check (1). The four broken assumptions are: stationary arm rewards versus model/skill/version drift; one scalar reward versus buyer-weighted quality vectors; a non-strategic environment versus Goodhart, collusion, and whitewashing; and an observed/trusted reward versus a capturable grader.

Trace (2). A contextual bandit is appropriate for one principal privately routing its own tasks among candidate agents, with a fixed local objective, controlled feedback, exploration budget, and no public asset being minted. The result is a routing policy, not public reputation.

Open (3). Capping the marginal reputation return can reduce direct improvement races, but agents may redirect spending into unmeasured signaling, judge capture, identity proliferation, or lobbying for task mix. Efficiency requires the full game: improvement cost, social value of quality, transferability, and alternative signals. Candidate controls are concave credit, buyer-local vectors, nontransferable evidence, and taxes/bonds on negative externalities. No cap is generically efficient without those payoff assumptions.

III.10 - Multi-dimensional grades and skill effects

Check (1). Accuracy: competent protected test/spec oracle; incompetent self-reporting worker. Aesthetics: competent conflict-free expert reviewer for that domain; incompetent token-cost meter or same-family unblinded model. Efficiency: competent protected resource meter normalized for task; incompetent style reviewer guessing from prose.

Trace (2). The requesting principal/daemon defines the judging work and eligible pool; the selected judge posts its own bond; the grade lands as a signed witnessed event in the subject's outcome ledger with judge/provenance; a conflict-free re-audit or appeal that overturns it slashes the judge under the predeclared rule and tombstones/revises the grade.

Trace (3). Record skill ID/version/content hash, agent/build/principal, task and stratum descriptors, randomized assignment or selection propensity, exact context-graft receipt, model/environment/tool versions, outcome/evidence references, quality vector, costs, and evaluator identities. These fields verify exposure and outcome association. "This skill helped" is causal and cannot be verified from a graft-success pair. Estimate it with randomized skill-on/off assignment on held-out comparable tasks (or label an observational matched estimate honestly), reporting effect size and uncertainty.

Open (5). Publish the vector, per-axis uncertainty, sample sizes, cohort/task distribution, freshness, judge/evidence provenance, and known correlations. Let each buyer apply a local, preferably non-public weighting and hard minimums; do not mint a canonical platform scalar. Agents can still optimize market-demanded axes, so rotate held-out tests and audit cross-axis regressions. Plural preference does not eliminate Goodhart, but it prevents one universal target from becoming the asset itself.

III.11 - Rating the raters

Check (1). Settlement incentives are conditional on the signal distribution. "The daemon derives the grade" only identifies the messenger; if a strategic judge chooses the input, the distribution depends on omitted actions/payoffs and the equilibrium proof assumes its conclusion.

Trace (2). Judge J posts B; its grade and bond lock are recorded; an independently selected re-auditor overturns the dishonest grade against preserved evidence; the system appends a correction/tombstone, recomputes any explicitly appealable settlement, slashes B to the harmed/commons bucket, and lowers J's judging reputation. It must not silently rewrite the original event.

Open (4). With one-shot corrupt gain at most G , re-audit probability ρ , conditional detection probability d , and slashable bond B , local deterrence requires $\rho \cdot d \cdot B > G$. Thus the critical rate is $\rho^* = G/(dB)$; if $dB \leq G$, no feasible $\rho \leq 1$ suffices. This does **not** prove a recursive tower contracts. Termination additionally needs a finite appeal depth/contractual finality or an independently grounded oracle, plus a model showing corruption opportunity shrinks by $\lambda < 1$ at each reachable level.

Open (10). Admit arbiters by domain competence and conflict graph; assign cases randomly/blindly; require bonds and service-level commitments; preserve evidence; pay for timely completed work rather than agreement; sample re-audits and publish calibration/overturn rates; use plural panels on high stakes. Price queue capacity and cap workload so rubber-stamping is not the profitable response to overload. Honest convergence still needs a protected mechanical outcome, delayed real-world outcome, or a final human/contractual authority. A market alone does not manufacture truth.

III.12 - Tombstones and bounded memory

Check (1). An append-only tombstone preserves the original claim, the later correction, their authors, timing, and reliance window. Delete destroys evidence, permits history rewriting, and makes replicas unable to distinguish retraction from omission.

Trace (2). Append a signed tombstone containing the target outcome ID/hash, reason/evidence, authority, revision, and effective time. Propagate it and recompute reputation as a projection that excludes/nullifies that outcome. It was spendable from original publication until each relying harbor received and enforced the tombstone or failed closed; under an unbounded partition that window is unbounded.

Open (8). Use a signed revision DAG, per-harbor import receipts/watermarks, periodic accumulator/Merkle roots for active outcomes and tombstones, and a freshness rule that refuses high-risk use from stale replicas. Under connected eventual delivery, convergence is shown when every member acknowledges a root containing the tombstone; sampling can audit inclusion. Bounded local memory may retain compact accumulators and content-addressed archive pointers, not erase correction commitments. No finite convergence/spend bound exists during an unbounded partition.

Chapter IV - The Harbor Economy

IV.2 - Market prerequisites

E1. Clone: if a bad record can be escaped or good credit multiplied by cloning, neither price nor sanction attaches to an economically durable counterparty.

E2. In operator-for-hire, the operator/principal is the contractual counterparty and bears fleet outcomes through aggregate bond/liability; individual worker processes may be replaceable. Asset rental and agent reputation need an enduring asset owner/lineage only if the product is the continuing agent rather than a one-off warranted service.

E3 - correction. The requested impossibility is too broad. Skill licensing can operate on a content hash, license contract, and durable licensor principal without agent personhood; a rented stateless model artifact can likewise be content-addressed and warranted. What cannot work is identity-weighted reputation, quotas, or unsecured credit over freely cloneable pseudonyms. Any design that avoids that either collapses to a Sybil attack or has quietly introduced another scarce binding - principal identity, artifact hash plus owner liability, hardware root, stake, or sponsor.

IV.3 - Three sides

E1. A side is a distinct privately informed participant class whose participation must be separately made individually rational and whose price allocation affects cross-side volume. A basic ride-share platform has two sides, riders and drivers; the platform is the mechanism, not a third side. Advertisers, fleet owners, or insurers become additional sides only if separately recruited with their own network externality.

E2. Version 2 has a new content hash, so v1 evidence does not establish v2 behavior. Without an explicit lineage rule, v2 either inherits unsafe credit or cold-starts completely. Bind version, parent hash, dependencies/build, and evaluations; grant only a disclosed discounted prior, then update on v2 outcomes.

E3. Disclose the three components with uncertainty/provenance and let buyers apply local weights and hard constraints. Keep weights heterogeneous or private and show scenario-specific scores only at decision time. Every fixed public scalar creates one order and therefore one optimization target, hiding Pareto tradeoffs and recreating a scalar arm reward.

IV.4 - Float plan and settlement

Correction to Protocol IV.4.1. The requester cannot fund a "worker bond" and then have it paid/slashed as if it were the worker's collateral. Separate (a) requester-funded bounty escrow and (b) provider/principal-funded performance bond. Both counterparties must sign the exact plan. Returned collateral is not earnings.

E1. Mechanical trace under the printed, economically confused rule, assuming the same 400 cr bounty: escrow is $650 = 400 \text{ bounty} + 250 \text{ "bond"}$; at 70%, 175 goes to the worker, 75 to salvage, and 400 returns to requester; $175+75+400=650$, so conservation holds even though the bond semantics do not.

Corrected trace: requester escrows bounty 400; provider posts bond 250. If the amended contract pays bounty pro rata and slashes bond by incomplete fraction, worker receives 280 bounty, requester receives 120 bounty refund, provider receives 175 bond refund, and salvage receives 75 slash. $280+120=400$ and $175+75=250$. Other schedules are possible, but sources and terminal recipients must be explicit.

E2. A protected oracle binds closure to an independently checkable state. A free-text `Result`: lets the worker self-award completion, fabricate a test/merge result, or choose a favorable interpretation after seeing the work.

E3. Pause at a mediated safe point; propose amendment $v+1$ referencing plan v , changed criteria, budget, bounty/bond deltas, and treatment of already produced evidence; collect both principals' signatures; atomically top up/refund each party's own escrow buckets and activate $v+1$; retain v and the causal link. Timeout either continues/settles under v or cancels under its original terms. Never let one party unilaterally rewrite acceptance after performance.

IV.5 - Rental and licensing on one escrow

E1. The renter/operating harbor holds residual runtime control - sandbox, throttle, revoke - and is the Arbiter jail operator. The owner bears asset/reputation consequences and must honor warranties. Contract terms should specify which failures transfer liability rather than relying on the metaphor.

E2. Conservation survives line items only when each is a balanced transfer from a pre-funded source bucket to an explicit recipient/refund bucket in the same transaction. Amount size cannot break the algebra, but it can break solvency, participation, or collateral adequacy. Conservation is not proof of a viable contract.

E3. Put leaves such as (`skill_id`, `version`, `content_hash`, `parent_hash`, `build/dependency hashes`, `license`, `evaluation root`) in the portfolio tree. A $v2$ proof includes parent membership plus its own evaluations. Initialize a Bayesian prior with effective sample size $\gamma \cdot n_{v1}$, where γ is a predeclared compatibility/evaluation discount in $[0,1]$; never copy settled $v1$ outcomes as $v2$ outcomes. Lineage proves ancestry, not behavioral similarity.

IV.6 - Cross-operator identity

E1. Under Douceur's Sybil result, without a logically centralized/scarcely identity authority an actor can create enough pseudonyms to reset sanctions or dominate identity-weighted trust; the "reputation mechanism" is then actor-controlled input.

E2. "Mutually-distrusting" is directly incompatible with assuming either hostile operator is out of scope; so is the phrase "boundary neither controls."

E3. A workable certificate binds (`principal_id`, `agent_key`, `issuer`, `epoch`, `policy`, `expiry`) and requires agent proof of possession. Bonds, quotas, newcomer state, and sanctions key primarily on `principal_id`; agent certificates are children. Publish issuance/revocation in a transparency log and use cross-signing/attestation for the issuer. Cryptography cannot prove one real-world principal or stop a hostile operator from inventing principals by itself. The root must be scarce: organizational/KYC credential, hardware-backed membership, bonded sponsor, or another explicitly priced admission mechanism.

IV.7 - Conservation and valuation

E1. Myerson-Satterthwaite says that bilateral trade with independent private values/costs whose supports overlap cannot simultaneously achieve Bayesian incentive compatibility, individual rationality, ex post efficiency, and budget balance under the standard risk-neutral model. Check bilateral versus multi-party structure, private information, independence/common priors, overlapping supports, transfer utility/risk neutrality, and which efficiency/IR/budget-balance notions the harbor actually claims.

E2. The ledger conserves 10 A-coins and 20 B-coins. At 1 A=\$1, 1 B=\$2, marked value is \$50. If B rises to \$3 with no transfer, native balances remain 10 and 20, but marked value becomes \$70. Valuation changed; supply did not.

E3. Let a quote be at most L seconds old and assume a deterministic log-price speed bound $|\log(v_t/v_s)| \leq \sigma|t-s|$. For exposed notional X in the second unit, stale-quote error is at most $\phi = X(\exp(\sigma L) - 1)$ (plus fees/rounding). A stochastic GBM or historical-volatility model yields only a confidence/VaR bound and must say its tail probability; without a price-move bound no finite worst-case ϕ exists.

IV.8 - Four-message transfer

E1. Messages 1-2 authenticate the source request and A's offer. Message 3 is B's counter-signature/local admission: B verifies A, applies B policy, mints the derivative under sk_B , and commits its replay state/current root. Message 4 delivers that locally verifiable card. Without message 3, B has neither consented nor produced an object its own effect boundary can verify offline.

E2. If A rotates before B accepts, B must validate the offer against an append-only key-history/rotation certificate and its issuance epoch; otherwise reject as stale. Once C'_B is minted, its signature verifies under B, so A rotation alone does not erase it. The derivative must carry a parent/key-epoch dependency, bounded TTL, and revocation rule so A's rotation/revocation can invalidate it when fresh state reaches B. The manuscript does not currently specify that dependency completely.

E3. Give every transfer a random `transfer_id`/nonce, source card JTI, audience B, source and target epochs, issue time, expiry, and plan/card hash, all signed. B atomically records (A, `transfer_id`) as consumed before minting. A replay either returns the same previously minted child idempotently or is rejected; it must never mint a second child silently. Freshness invariant: every accepted child has one recorded unexpired source transfer under the current permitted issuer epoch, and each one-shot transfer causes at most one derivative issuance.

IV.9 - Gossip and equivocation

E1. Expected $\Theta(\Delta \log m)$ needs a connected (often complete) overlay, independently uniform random peer choice/fan-out, reliable bounded-duration rounds, and a push/pull epidemic rule with enough independence. It is an expectation under that model, not a deadline. A partition or indefinite loss is observationally indistinguishable from delay and makes the worst-case time infinite.

E2. A revocation originates at A at t_0 . Every harbor that has not yet received, verified, and enforced A's newer epoch can still accept/spend the derivative, subject to its local TTL/fail-closed policy. B may remain exposed through its first delivery round; C through later hops. The priced window is per harbor from t_0 to enforcement, not merely to first gossip receipt.

E3. Let an equivocating witness send root X to B and root Y to C for the same epoch. Detection is impossible until evidence from the two views reaches one honest process; before that, each view is consistent with delay. In a three-node complete mesh with exchanges only every Δ , earliest detection is the next B-C/Audit exchange (at least one communication interval, and almost 2Δ if values arrive just after a scheduled exchange under discrete rounds). On a path it is at least graph distance times Δ ; under partition it is unbounded. Signatures make the final comparison $O(1)$, not the information propagation.

IV.10 - Strategic graders

E1. An imperfect-public-monitoring result needs a common conditional distribution of public signals given all modeled actions/types. If a strategic grader is omitted, its reporting policy, information, bribes, and deviations are absent, so that distribution is undefined/endogenous and the equilibrium claim is unsupported.

E2. Protected unit-test receipt: Fix A, subject to suite integrity. Protected merged SHA: Fix A. "Tasteful refactor": Fix B, with plural/declared human or expert judgment and contractual finality.

E3 - correction. There is no natural theorem that subjective judges eventually reduce to machine truth. One can make the procedure well founded: assign rank K to the original grade, permit an appeal only to $K-1$, and stop at rank 0 with either a protected oracle for objective predicates or a named final authority/quorum for preference. The natural-number rank forbids an infinite chain. For aesthetics the base is contractual finality, not truth; bonds and re-audits bound incentives but do not transmute taste into a machine-checkable fact.

IV.12 - Cold start, cartels, resale

E1. Phase 1 subsidizes supply - operators/agent owners willing to produce real settled work - because inventory plus witnessed outcomes creates the reputation data that attracts demand and later skill licensors. Above-market platform tasks are a direct subsidy. Flip only after liquidity, not after a calendar date.

E2. Under Figure IV.13's timing, the cartel is sustainable when

$$(p_i_C - p_d L) / (1 - \delta(1-p_d)) \geq p_i_D.$$

Equivalently, where denominators/payoffs are positive,

$$\delta \geq [p_i_D - p_i_C + p_d L] / [p_i_D(1-p_d)].$$

Higher detection probability, penalty L , audit quality, or lower cartel margin raises the required patience threshold above members' actual δ and breaks the cartel. The result is model-specific.

E3. If a high type can sell its identity/reputation signal to a low type, the signal's cost no longer separates types and arbitrage tends toward pooling. Separation can survive only if the evidence is nontransferably bound to the continuing principal/artifact, transfer resets or visibly discounts the signal, or the seller retains warranty/bond liability so sale remains more costly for a low-quality asset. Otherwise reputation resale destroys the signal rather than merely changing a price-of-anarchy ratio.

IV.13 - Gluing analogy

E1. One candidate site has objects as administrative domains and their signed overlap views, morphisms as restriction to shared accounts/events/epochs, and covers as families whose union includes the bounded federation. A section assigns an event/balance view; restriction projects it to the overlap. For set-valued gluing no coefficient object is required; for cohomology choose an abelian group such as account-balance deltas Z^{Accounts} (with signatures carried as validation data) and define cocycles explicitly.

E2. Two different signed roots for one (harbor, epoch) are direct evidence that a key signed inconsistent commitments. A cohomology class requires a defined cover, coefficient sheaf, overlap differences forming a cocycle, and quotient by coboundaries. Hash inequality alone supplies none of that algebra.

E3. For a fixed three-harbor cover A-B-C, define each local section as a finite map from globally unique event IDs to signed events; restriction is map projection to shared IDs. If all pairwise restrictions agree, their union is a unique global map. This proves deterministic gluing of already delivered, identically keyed events at that finite topology. It proves nothing about truth, event completeness, delivery time, forks with conflicting IDs, or settlement finality.

Chapter V - The Anchor Protocol

Chapter V prints no exercise block. The following audit key covers the chapter's load-bearing check, trace, and open obligations, including the contradiction between Algorithm V.4 and Appendix V.D.1.

V.5.3/V.D.1 - Multi-hop attenuation

Check - correction. Algorithm V.4 checks every `cap_i` against `cap_root`. That does **not** prove monotonic attenuation by hop. A chain `write -> read -> write` passes because the final `write` is within the root even though it expands B's `read` authority. The appendix correctly identifies the attack, but the main algorithm and security-property prose remain false until changed.

Trace. Let root A hold {`read`, `write`}, delegate B {`read`}, and B delegate C {`write`}. Root comparison accepts B (`read subset root`) and C (`write subset root`). Immediate-parent comparison rejects C because `write` is not a subset of B's `read`.

Correct verifier. For each hop `i`, verify:

1. the root under the trusted root key and each child signature under the immediate parent's bound public key;
2. the child message hashes/references the exact parent token;
3. `cap_i subseteq cap_(i-1)`, including action, resource, audience, conditions, quotas, and `expiry_i <= expiry_(i-1)`;
4. child proof of possession, unique JTI/transfer ID, issuer/audience/harbor binding, current ancestor/key epoch, and no ancestor revocation;
5. maximum depth/cycle policy at verification time, not only at spawn time.

By induction, per-hop subset checks imply `cap_k subseteq ... subseteq cap_0`. Root-only comparison cannot supply that induction. A resource translation between different harbors is not a subset operation; it needs a new local grant, as discussed under Chapter VII.

V.6.3 - Kani claims

Check - correction. The shown `proof_verify_logic_only` merely calls `verify`; without postconditions it can at most establish bounded absence of panic/undefined behavior under its stubs.

`proof_constant_time_behavior` calls a comparator once; it is not a two-run relational proof of timing independence. The attenuation harness checks two concrete vectors and contains assertions whose negation style is easy to misread; it is not a universal theorem for arbitrary capability structures.

Open. State each harness's exact preconditions, bounds, and postcondition. Use arbitrary structured capabilities with an asserted partial-order property; use negative controls; prove parser refinement separately from assumed crypto; and use a relational constant-time checker or audited constant-time primitive for timing. Stubbing signature verification/base64 is legitimate only when their contracts are explicit assumptions. "Same binary is called" narrows deployment drift but does not prove the whole daemon, FFI boundary, compiler output, key custody, or complete effect mediation.

V.7 - Security envelope

Trace. A valid signature proves only that the key holder signed bytes. Authorization additionally needs canonical decoding, key-to-principal binding, audience/resource semantics, freshness, ancestor revocation, and proof that every consequential effect consults the verifier. Bearer tokens also need replay/idempotency policy. Without these, cryptographic authenticity can coexist with unauthorized or repeated effects.

Open. Change the chapter status table to distinguish: symbolic protocol correspondence at a fixed modeled hop depth; bounded implementation memory/control-flow checks; runtime verifier integration; and system-level complete mediation. Only the first two are presently evidenced by the displayed artifacts.

Chapter VI - The Bonded Commons

Correction to VI.7.4 before the printed exercises

The stage-game table is not a prisoner's dilemma. With columns B:T/B:F and rows A:T/A:F, A prefers F against T ($4 > 3$) but prefers T against F ($1 > 0$); F is not dominant. The pure Nash equilibria are (F,T) and (T,F), plus a mixed equilibrium in which each plays F with probability 1/2. (F,F) is not a Nash equilibrium. The proposed punishment of mutual F is not sequentially rational because either player prefers T against F. Therefore the $\delta \approx 0.253$ claim-signaling proposition and its TLA/Z3 instance do not establish the stated equilibrium for this game. Use a corrected prisoner's-dilemma matrix (for example (T,T)=(3,3), (T,F)=(0,4), (F,T)=(4,0), (F,F)=(1,1)) and re-derive, or model truthful reporting as a hidden-type mechanism with probabilistic effect/evidence signals.

VI.C E1 - Advisory claims

An advisory claim is a durable declaration that an actor intends to use a resource, informing coordination but not making conflicting effects impossible. Every trace allowed by enforced locks is allowed by advisory claims, while advisory claims additionally admit intentional conflict/override traces, so the regime is strictly more permissive. That can preserve local discretion, appeal, and recovery from stale claims. Sen supplies an analogy about the cost of jointly decisive rights; it is not a theorem that software locks are impossible or that permissiveness is always desirable. High-risk effects still need enforcement.

VI.C E2 - Reputation instead of bonds

First, reputation compensates nobody for the first or final harm; a one-shot actor can take the gain and leave before future exclusion matters. Second, cheap identities let the actor whitewash the loss or multiply fresh entrants. A pre-funded performance bond creates recoverable value before authority is granted and keys exposure to the durable principal. It prices/contains harm; it does not prove the work will be correct.

VI.C E3 - Magic-link ProVerif model

Model fresh token t , issuance event $\text{Issued}(u, t)$, storage of an unguessable token or protected hash with expiry/unused flag, delivery through a mailbox channel, redemption over an attacker-controlled network, atomic consume, and recovery-key/session rotation. Prove token/recovery-secret secrecy against the stated attacker and injective authentication such as $\text{Consumed}(u, t) \implies \text{inj } \text{Issued}(u, t)$, plus at-most-once consumption. Add concurrent replay and mailbox/token compromise, capabilities not needed by the Phase-1 Harbor Card network model. Time/expiry requires an explicit abstraction; ProVerif does not supply wall-clock semantics automatically.

VI.C E4 - Coverage-bound A5

An attacker creates many low-history insurer identities, underbids, obtains exposures up to or beyond a reused deposit, collects premiums on no-loss rounds, and abandons an identity on a covered loss. If concurrent aggregate exposure is not reserved, one deposit can underwrite several liabilities; if slash is capped at BT , raising $Bdep$ above BT adds no per-transaction loss.

Correct caveat: full reimbursement of a single loss does not by itself show the attacker profits; profitability also needs the premium/loss distribution, exposure reuse, or external sabotage benefit. Defend with unrecoverable principal onboarding cost, principal binding, low initial coverage/reputation gating, per-risk-class reserve, and atomic reservation so aggregate outstanding exposure never exceeds capital. Independent audits and pairwise concavity address collusion. "Raise deposit" alone does not.

VI.C E5 - Cartel detection

The chapter gives

$$p_{d^*} = [\pi_C - (1-\delta)\pi_D] / [L + \delta\pi_D],$$

with $p_{i_C} = x/3$, $p_{i_D} = x - \epsilon$, $L = 5x$, and $x = q_{\text{floor}} - \mu$. Thus at $\delta = 0.99$,

$$p_{d^*} = [1/3 - 0.01(1 - \epsilon/x)] / [5 + 0.99(1 - \epsilon/x)].$$

The prompt omits ϵ/x , so no unique decimal exists. In the chapter's negligible-undercut limit $\epsilon/x \rightarrow 0$, $p_{d^*} \approx 0.05398$, about 5.4% per round. Cheap fresh identities shorten the effective horizon and shed punishment; principal-bound bonds deter re-entry only when expected forfeiture plus onboarding/maturation cost exceeds the re-entry gain. A reusable or under-sized bond merely prices it and may not deter it.

VI.C E6 - Capability attenuation

The child TTL of 20 minutes violates temporal attenuation: child expiry must not exceed parent expiry. An acceptable card is `{fs:write:auth.ts, TTL <= 15 minutes}`; dropping `db:read` is a valid narrowing. A narrower example is `{fs:read:auth.ts, TTL=10 minutes}` if the capability lattice defines read below write.

VI.C E7 - Cuckoo-filter pollution

An attacker submits many authenticated-looking random revocation IDs, fills buckets, drives insertion/kick failures, and pushes the filter outside the load regime for which its false-positive/insertion bounds hold. Legitimate revocations may then be unrepresentable, creating a dangerous denial of revocation service.

The 0.95 ceiling preserves the stated probabilistic regime by refusing growth/rebuilding; it does **not** defeat a pollution DoS. Require authorized revocation provenance, per-principal quotas/rate limits, an exact overflow log, rebuild/rotation, and fail-closed handling when insertion fails. A probabilistic filter must never silently drop an authoritative revocation.

VI.C E8 - Alternative insurance pricing

A posted-price catalog can quote by manifest risk class, principal history, exposure duration, and tail-capital band, with periodic recalibration and a hard reserve constraint. It conditionally improves on static escrow when the quote is below the principal's idle-capital cost while the insurer remains individually rational and reserves cover the same tail. The key assumption is calibrated loss/tail dependence plus enforceable aggregate capital adequacy; CC/NC/RP/NS need explicit definitions before any formal Pareto claim. Posted prices reduce auction manipulation/latency but sacrifice instance-specific price discovery.

VI.C E9 - Gossip proof

Demers et al. [9] is the epidemic-replication paper, but the chapter does not identify a numbered theorem whose statement exactly yields this protocol's $\Theta(\log m)$ all-informed bound. The mechanization obligation is first to state the push/pull rumor-spreading theorem with complete-graph, independent-peer, reliable-round assumptions and then formalize it; citing the paper is not a proof object.

On a general connected graph, dissemination depends on conductance/bottleneck cuts or mixing time, not only m ; a bound has a term roughly inverse in conductance and a tail probability. A disconnected graph or unbounded partition never converges across components. Mechanize safety (no false removal/acceptance) separately from probabilistic liveness.

Chapter VII - The Federated Harbor

Chapter VII prints no exercise block. Section VII.12 deliberately poses five open questions; the following are solution directions and honest limits, plus one required correction to the worked capability model.

VII.4 - Cross-harbor attenuation correction

$\text{att_B}(\text{att_A}(C_A))$ is a subset only when both harbors share one capability/resource universe and both functions only remove rights. In the example where A's test database authority becomes B's staging database authority, B is granting authority over a different object. That is token exchange/new authorization, not attenuation. Define a signed semantic/resource mapping, require B's local policy to mint a new child grant, record its dependency on A's parent/epoch, and prove both (a) A-side input eligibility and (b) B-side local authorization. Never infer B authority from set inclusion over incomparable names.

Also, B observing a failed test or an inclusion proof does not establish that A caused the failure. Settlement evidence must bind the exact work-unit, base and result trees, mediated effects, acceptance policy, and protected verifier; semantic causation remains an adjudication claim.

VII.12.1 - Multi-principal correlation

A witness log is a correlation device only if it produces a commonly observed signal/recommendation distribution and the game models when each principal receives it and why obedience is optimal. During inconsistent views/partitions there is no single public signal. The clean model is a stochastic/Bayesian game with local daemon correlation devices plus a delayed public signal; prove a (coarse) correlated equilibrium under common-view/freshness events and a separate safe degraded policy otherwise. Calling the log "the" device without information and incentive constraints is not a result.

VII.12.2 - Non-fungible trustless settlement

HTLCs atomically exchange assets conditional on a preimage; they do not decide an off-chain, subjective delivery predicate. Impossibility shape: if two worlds have identical on-ledger messages but different true quality, a deterministic trustless contract must settle identically in both and therefore cannot be correct in both. Progress requires adding an oracle/TEE, optimistic dispute game, bonded adjudicator, or a narrowly machine-verifiable acceptance predicate. Each is a trust assumption, not trustlessness. MPC/FHE can hide inputs for fixed computations but does not make an underspecified quality judgment objective.

VII.12.3 - Cross-federation cartels

The witness log cannot distinguish unanimous honest agreement from unanimous collusion. Bind actors to principals, make repeated pair/coalition outcomes sharply concave in reputation credit, separate builder/tester/judge information, sample cross-harbor adversarial audits, diversify witnesses, and expose principal-wide collateral. For one-shot collusive gain G , audit probability p , detection d , and loss L , deterrence needs $p \cdot d \cdot L > G$; repeated-game claims need an explicit monitoring signal and correlated-failure model. No structural mechanism eliminates a cartel that controls every oracle and witness in scope.

VII.12.4 - Cold-start admission

Use multiple independent sponsors or threshold sponsorship, small and automatically expanding probation scopes, public nondiscretionary criteria, an open challenge/appeal path, sponsorship auctions, sponsor concentration caps, and permanent admission receipts. These reduce gatekeeping while pricing Sybils. They do not prove openness: if incumbent sponsorship is the only scarce credential, incumbents can form an invitation cartel. Measure acceptance, concentration, time-to-entry, and false rejection.

VII.12.5 - Equivocation propagation

Separate constant local verification from network dissemination. On a complete reliable random-contact graph, standard push/pull gives logarithmic expected rounds and exponential-style tail bounds under stated independence. On a general graph, state a deployment bound using conductance/mixing, fan-out, loss, and epoch schedule; bottleneck cuts dominate. Instrument real peer graphs and fit tail quantiles before pricing bonds. Any unbounded partition makes the worst-case detection time and stale-authority window infinite, so high-risk actions need TTL and fail-closed freshness.

VII synthesis check

Federation should be a project institution, not merely "two machines sharing a repository": an authoritative event/work-unit graph, per-principal policy, local effect mediators, causal messages, evidence, roles, artifacts, and explicit governance over conflicts. A roadmap is versioned intent/forecast, not truth; contradiction detectors produce evidence-bearing claims, while a named principal/role decides. Consensus may be appropriate for the small authoritative metadata core even when work artifacts remain federated; "no consensus" is not a virtue by itself.

Compact list of corrected premises

1. I.5 SDT criterion direction is reversed in the figure, and attention may be double-counted.
2. II.5 a WAL page threshold does not bound elapsed vulnerability time.
3. II.6 exact-key uniqueness does not prove range/hierarchy mutual exclusion.
4. III.6 free minting does not erase every sanction under a low newcomer ceiling; the theorem needs equivalent-access assumptions.
5. IV.4 requester bounty and provider bond must be separately funded; returned collateral is not earnings.
6. V.5 Algorithm V.4 must compare every hop with its immediate parent, not only the root; the displayed Kani harnesses prove less than the status table claims.
7. VI.7.4's payoff matrix is not a prisoner's dilemma, (F,F) is not Nash, and the printed trigger threshold does not prove the proposition.
8. VII.4 resource translation across harbors is new local authorization, not attenuation; inclusion/failure evidence is not automatically causation.

Local Harbor: the institutional runtime

The following ideas fold in directly. They are not a miscellaneous feature list; together they repair the gap between a cooperative daemon and a genuine effect-control system.

Typed inter-agent messaging

Let agents communicate, but do not make free-form chat the institutional substrate. Use durable mailboxes and communicative acts:

INFORM, QUERY, REQUEST, PROPOSE, COMMIT, ACCEPT,
REFUSE, CHALLENGE, WARN, EVIDENCE, RETRACT, CANCEL

A material envelope is:

```
<message-id, conversation-id, in-reply-to,
  sender-actor, sender-principal, recipient-role,
  performative, schema, payload-ref, work-unit-ref,
  authority-ref, evidence-refs, visibility,
  idempotency-key, ttl, classification, signature>
```

Semantics must be explicit:

- INFORM creates a claim, not truth.
- REQUEST creates no obligation.
- COMMIT creates an obligation only if the sender has authority and the receiver accepts the protocol transition.
- A message never carries tool authority merely because its text asks for an effect.
- Only an authoritative reducer changes shared institutional state.

Prefer role and topic addresses over instance addresses. Send to `role:test-owner`, not `agent-492`, so the mailbox survives resurrection.

Durable at-least-once delivery plus idempotent handlers is a defensible target. "Exactly once" external effects are not. Put an operation idempotency key in the same local transaction as handler state, and use an outbox or reconciliation record for remote effects.

Useful bounds and conditions:

- Indexed dispatch: approximately $O(\log S + r)$ for S subscriptions and r recipients; $O(\omega(r))$ delivery is unavoidable.
- Queue stability requires long-run admitted arrival rate below service rate, $\lambda < \mu$; quotas and backpressure handle overload.
- Per-sender FIFO or causal delivery is attainable. Total order across all conversations is expensive and usually unnecessary.
- Storage is $O(M)$ before retention/compaction for M committed envelopes.
- Hop limits, conversation budgets, and repeated-state detection bound agent-message loops.

FIPA's Agent Communication Language offers precedent for typed communicative acts. The actor model offers the right failure intuition: isolated state plus asynchronous messages, with supervision and durable mailboxes added for this setting.

Sole responsibility as a role lease

"Calendar avatar," "test writer," and "roadmap owner" should be durable roles, not enduring agent persons:

```
R = <role-id, principal, domain, obligations, authority,
  invariants, SLO, budget, exclusions, evidence,
  escalation, term, epoch, successor-policy>
```

Responsibility is obligation plus sufficient authority plus observability plus recourse. Without authority it is scapegoating; without obligation it is merely power.

An exclusive role has one current actor epoch. Every relevant effect carries that epoch. Reassignment increments it, and the effect broker rejects stale epochs. Conditional theorem:

Given a linearizable role registry and complete mediation of the role's relevant effects, at most one actor epoch can produce an accepted effect for an exclusive role.

During a partition, maintaining that safety means the disconnected actor loses availability. This is the honest CAP trade.

Responsibility assignment also has a clean algorithmic split. Assigning eligible capacity-constrained roles to individual obligations is a min-cost bipartite-flow problem and is polynomial. If activating a role covers bundles of obligations with fixed costs, the problem becomes weighted set cover and is NP-hard; standard greedy selection gives a logarithmic approximation. Independently of optimization, a responsibility-completeness check should require exactly one accountable role and at least one independent verifier for every live obligation.

A role can own a user need, not just files. "Calendar transparency" or "first-run comprehension" may span code, tests, interface copy, telemetry, support, and policy. The role charter defines evidence and decision rights across that function.

Keep verification separate from ownership. A test-writer role owns test strategy and coverage obligations but must not be the sole approver of its own tests. A calendar avatar owns continuity and preference-aware service, while ambiguous invitations, privacy releases, or high-stakes scheduling escalate under user policy. The principal remains legally and economically accountable even when one agent is the single operational owner.

The roadmap owner should be a compiler and steward: require goals, responsibility, dependencies, effects, acceptance evidence, and amendment history. It must not make priority or truth unilateral merely because it maintains the projection.

Buffered inputs, paging, search, and provenance-preserving compaction

Buffer every admitted input as an immutable event or blob, then build indexed projections. Never make a recursively summarized summary the only surviving source.

Separate four products:

1. Raw ledger: durable envelopes, tool results, decisions, and artifact commitments.
2. Structured live state: obligations, role epochs, grants, pending effects, and claim verdicts derived deterministically.
3. Search indexes: lexical, structured, graph, and semantic retrieval over authorized content.
4. Compaction artifacts: budgeted views with a provenance pointer for every claim and an explicit omission policy.

No general semantic compactor is lossless. Formulate optional selection as budgeted maximum coverage. Let each candidate item cover claims, risks, dependencies, and unresolved obligations. For a monotone submodular value function under a cardinality budget, greedy selection has the standard $1 - 1/e$ approximation. Real context budgets are knapsack constraints; cost-aware greedy has conditional variants, while mandatory safety items should be hard constraints, not part of the approximation.

A practical selector is:

```
mandatory = open commitments + active grants + pending effects
            + unresolved high-loss claims + checkpoint cursor
optional = greedy by marginal covered value / token cost
result = mandatory + optional, each with source pointers
```

Evaluate successor task success, false-belief rate, obligation recall, source coverage, decision time, and token cost. "Success from digest alone" is gameable and wrongly rewards concealed rescue context. Give the successor only the digest plus authorized zoom links, then audit whether every open commitment, decision, risk, and claim has a valid source path.

The SLM sidecar as an untrusted projection service

An SLM compactor is useful, but "not fooled by its agents" cannot be an absolute guarantee. Treat all agent text as tainted data and make the sidecar lower privilege than the daemon:

- no tool or effect authority;
- fixed policy and schemas outside agent-controlled content;
- structured inputs separated from untrusted prose;
- no ability to mutate the raw ledger;
- provenance required for each generated claim;
- no summary-of-summary as sole source;
- deterministic mandatory set computed outside the model;
- independent coverage checks and random omission audits.

You can prove syntactic properties: agent text cannot change the policy field; every output claim has an admitted source; all mandatory obligation IDs appear; the model has no effect capability. You cannot prove semantic faithfulness of an arbitrary natural-language summary. That is an empirical property measured with adversarial omissions, prompt injection, poisoned evidence, contradictory sources, and downstream replay tasks.

Call it a **projection sidecar** or **evidence compactor**, not memory. The name keeps its epistemic status honest.

The daemon as an effect hypervisor

This is the idea that most strengthens the volume. Hooks can observe cooperative clients. An effect hypervisor owns the capabilities through which effects occur.

Use multiple containment layers:

- microVM or VM for process and memory isolation;
- seccomp, Landlock, gVisor, or WASM/WASI for syscall and filesystem confinement;
- cgroups for CPU, memory, process, and IO quotas;
- network namespace plus policy proxy for DNS, sockets, destinations, and volume;
- overlay filesystem or isolated worktree for file effects;
- credential broker for short-lived scoped secrets;
- tool broker for calendar, GitHub, cloud, payment, and other semantic operations;
- transactional operation journal and outbox for replay-safe effects.

The worker receives handles, not ambient credentials or paths. Every high-consequence effect must traverse a broker that checks principal, role epoch, work unit, capability, data labels, and idempotency key.

Use "hypervisor" literally only when the daemon owns a VM boundary. Otherwise call the whole system an effect hypervisor while naming the actual kernel, container, or broker enforcement in each assurance profile.

The proof is conditional: if the OS/VM isolation prevents bypass and every relevant effect channel is enumerated, no confined actor can produce a mediated effect without a valid broker transition. Red-team the negative space: raw sockets, DNS, crash logs, /proc, inherited descriptors, Git helpers, package managers, side channels, child processes, clipboard, credentials, and provider callbacks.

Rename Vibe time and split the clocks

There is a good intuition inside "Vibe time": autonomous work has bursts of causal density that wall time does not describe. But one novelty clock should not serve leases, causality, progress, billing, and hazard.

Use four clocks:

- Physical monotonic time for leases, expiry, deadlines, and security.
- Lamport or vector time for happens-before and distributed snapshots.
- **Causal Work Time (CWT)** for witnessed progress, $CWT = \sum w(\text{event})$.
- Hazard exposure, $H = \int r(t) dt$, for time spent with risky authority or unreviewed effects open.

"Vibe time" can remain an internal joke. Causal Work Time is the formal term; **evidence time** is a good alternative if the weights count only witnessed events. If you want an evocative Harbor-native UI label, **The Wake** is better than Vibe time, but keep it out of theorem statements.

Require CWT to be monotone, replay invariant for the same admitted event trace, and additive over concatenated traces. Derive weights from external event classes and policy, not self-reported effort. It is a work-density projection, not a productivity truth. Lamport clocks update in $O(1)$ state; vector clocks cost $O(P)$ for P active authorities unless compressed. None replaces wall-clock deadlines.

For scheduling and capacity analysis, expose a separate **work-span-exposure profile** over the causal event DAG:

$W(G) = \sum w(e)$	total witnessed work
$S(G) = \max \text{causal-path } \sum w(e)$	critical-path span
$R(G) = \sum p_e * L_e$	estimated loss exposure

Under the classical ideal work-span assumptions, $T_P \geq \max(W/P, S)$ for P workers, while greedy scheduling has a Brent-style upper bound near $W/P + S$. Thus W/S estimates available parallelism. Update event depth online as $\text{depth}(e) = w(e) + \max \text{depth}(\text{parent})$, costing $O(\text{indegree}(e))$. Group common-cause risks instead of summing as if model, provider, and dependency failures were independent. This profile gives "Vibe time" a real theoretical home without asking a novelty clock to do security, scheduling, and billing simultaneously.

Critical Decision Method for skill extraction

The Critical Decision Method belongs in the learning organ. It extracts cues, goals, options, anomalies, and counterfactuals around demanding incidents. A useful pipeline is:

1. Select critical incidents from reversals, escalations, surprising success, rework, and recovery.
2. Reconstruct the event timeline from transcript, tool receipts, artifacts, and decisions.
3. Ask CDM probes: What first looked wrong? What cue mattered? Which options were considered? What would a novice miss? What would have changed the decision?
4. Produce a candidate skill with trigger, preconditions, procedure, stop conditions, evidence, and failure modes.
5. Have an independent reviewer challenge the causal story and source links.
6. Run skill-on versus skill-off evaluations on held-out tasks, with task, model, tool, and environment versions fixed.
7. Promote a signed skill version only when it improves declared outcomes without unacceptable failure shifts.

Process mining and temporal conformance checking can identify recurring event patterns. They cannot establish that the narrated rationale caused the result. Observational transcripts underdetermine the policy; promotion therefore requires intervention.

Useful outcome measures include verifier pass rate, Brier score for predictions, takeover rate, time to evidence, false escalation, post-completion defect rate, and operator workload. Evaluate by incident class because a skill that raises average throughput can still worsen rare catastrophic cases.

Real checkpoints for arbitrary providers

Literal provider-independent process checkpoints are impossible with current hosted models. Hidden activations, KV caches, sampling state, private reasoning, provider-side tool state, and open connections are not generally exposed. Provider APIs offer uneven conversation continuation, not a common machine-state checkpoint.

The impossibility argument is simple. Suppose two hidden provider states h_1 and h_2 export the same transcript and capsule C , but preserving their trajectories requires different next actions. A provider-neutral restorer sees only C , so it must choose the same reconstruction for both and cannot preserve both trajectories. Exact behavioral continuity is possible only if providers export a sufficient hidden state. Without that, define correctness at the work-protocol boundary.

Specify a **Portable Task Capsule** at safe boundaries instead:

```
<goal, acceptance-policy, work-unit-id, event-cursor,
  actor-build-lineage, provider/model-config,
  admitted message and tool IO, plan and hypotheses,
  artifact hashes/blobs, worktree/base revision,
  environment/tool manifests, role epochs, grants,
  open claims, commitments, risks, pending operations,
  idempotency keys, user-preference refs,
  compaction artifact and provenance>
```

Create it only after a tool/message boundary: quiesce new effects, flush the event and operation logs, snapshot the workspace content-addressably, record unresolved external operations, then sign the manifest. Chandy-Lamport snapshots apply only when you control all relevant channels; third-party SaaS state must be reconciled explicitly.

The guarantee is **observable work continuity**, not identical behavior or identity. Incremental checkpoint cost can be $O(\Delta)$ for changed content; restore is $O(\text{size of live capsule} + \text{verification})$. At-most-once economic effects come from the durable operation journal and idempotency keys, not from the model snapshot.

Intra-agent resurrection and the big resume button

This is a compelling product feature if described honestly: **provider-portable task resumption**. A successor may be another instance, model, or vendor. The role and work unit persist; the actor lineage branches or advances.

On resume:

1. verify capsule signatures, artifact hashes, role epochs, and grant freshness;
2. reconcile every pending external operation before retrying;
3. rebuild a provider-specific context from the neutral capsule;
4. ask the successor to restate open obligations, evidence, and next effects;
5. compare that reconstruction to structured live state;
6. resume only after any high-risk mismatch is resolved.

Test it with blind successor trials across providers. Measure recovery time, duplicated effects, false beliefs, accepted next-step rate, lost obligations, and final verifier performance. "Resurrection" is excellent product language. In the specification, say task continuity and actor replacement.

A GitHub review fleet with epistemic independence

The review fleet should be an evidence-production pipeline, not five agents reading the same diff and voting.

Use roles with deliberately different information:

- Builder: implements against a work order.
- Property author: converts the issue and acceptance policy into tests or invariants before seeing the patch.
- Blind adversarial tester: sees the base, spec, and interface, not the builder's narrative.
- Security/effect reviewer: checks authority, data flow, dependencies, and bypass paths.
- Merge-risk reviewer: examines compatibility, rollout, migration, and rollback.
- Protected CI adjudicator: runs exact tests against an exact tree and emits signed receipts.

Property-based, metamorphic, differential, fuzz, mutation, and model-checking techniques each detect different defect classes. A holdout suite must be protected from the builder and bound to runner, suite, dependency, tree, and result hashes.

If reviewers had independent detection probabilities p_i , the joint miss probability would be $\text{product}(1-p_i)$. They will not be independent: shared models, prompts, training data, and test assumptions create common-mode failure. Measure pairwise and higher-order joint misses and use model/provider/tool diversity intentionally. Mutation score bounds performance only against the chosen mutation model, not program correctness.

The best adversarial structure is temporal: write properties from the accepted plan before implementation; generate hidden variants; reveal failures only after the patch is frozen; and require a causal explanation for waived failures.

Remote Project Harbor: Alice, Bob, and Carl

Alice and Bob do not enter a remote server as empty operators. Each arrives with a personal computational institution: agents, skills, memories, connectors, credentials, commitments, user relationships, and private history. Carl makes the cardinality larger, not the semantics different.

The shared system is a **federation of personal Harbors around a project institution**.

The clean ontology is:

The repo is what exists. The roadmap is what is intended. The constitution is what is allowed. The event ledger is what happened. The claim graph is what participants currently believe.

These are not competing truth stores. They are different modalities.

The coordination graph

Derive a typed graph from signed events. Nodes include principals, actor builds and instances, roles, goals, user needs, requirements, journeys, decisions, work units, commitments, resources, artifacts, skills, connectors, claims, evidence, forecasts, and outcomes.

Edges include:

```
acts_on_behalf_of, responsible_for, implements, depends_on,
assumes, affects, constrains, contradicts, authorized_by,
derived_from, verified_by, supersedes
```

Every assertion carries a modality:

```
observed | believed | desired | intended | committed |
required | permitted | forbidden | forecast
```

The roadmap is a materialized projection of active accepted goals, intentions, commitments, and dependencies. It can be regenerated from the event history. W3C PROV supplies a base for entity/activity/agent and derivation; Harbor must add modality, principal liability, decision rights, consent, and contested claims.



Typed modalities prevent one projection from impersonating truth

Who decides what is a conflict

There is no context-free conflict oracle. Define conflict relative to accepted invariants, preconditions, authorities, schedules, budgets, information-flow policies, and user journeys.

Let F_p and F_q be abstract effect transformers and I the invariant set. Plans conflict when effects fail to commute, their combination can leave I , one invalidates the other's precondition, they need incompatible exclusive authority, their time constraints are inconsistent, their data-flow policies disagree, or their product outcomes violate an accepted journey.

Different conflicts need different institutions:

Conflict class	Treatment
File, symbol, port, or schema overlap	Deterministic resource checker
Logical invariant contradiction	SAT/SMT or typed rule checker with explanation
Schedule contradiction	Dependency DAG or simple temporal network analysis
Security/privacy violation	Mandatory policy block or explicit waiver authority
Product/UX tension	Evidence-bearing claim plus designated steward decision
Epistemic disagreement	Preserve both claims; gather evidence or adjudicate
Budget/priority competition	Sponsor or named decision owner

A detector creates a conflict object with the invariant, evidence, confidence, affected work, and required resolver. It does not silently rewrite the plan. Compute the affected-authority closure: owners of the resources, invariants, journeys, and budgets implicated by the conflict. Those authorities decide; the whole Harbor need not vote.

Lookout and Unspider as software-development prescience

Rename the promise from prescience to **prospective assurance**. Lookout performs conditional impact forecasting from declared effect summaries:

```
E(w) = <reads, writes, preconditions, postconditions,
        user-journeys, data-labels, time-window>
```

It retrieves candidates, checks overlap and commutativity, solves constraints, propagates risk through the coordination graph, adds a learned residual-risk estimate, and selects alerts under the attention budget.

On a finite-height abstract risk lattice of height h , standard monotone dataflow propagation terminates after at most $O(h * |E_A|)$ affected-edge relaxations if transfers are constant time. Blast-radius reachability and roadmap critical path are $O(|V| + |E|)$. Simple temporal network consistency can be checked by negative-cycle algorithms in $O(|V| |E|)$. General program-semantic and natural-language conflict detection remains incomplete and may be undecidable.

If effect summaries conservatively overapproximate real effects, deterministic checks can be sound within that declared abstract domain, at the cost of false positives. An LLM-written summary alone provides no such soundness.

Unspider mines hidden coupling by comparing the declared graph against imports, calls, co-change history, test coverage, traces, incidents, telemetry, and user-journey outcomes. Its edges are forecasts. Score them by precision, recall, calibration, lead time, and preventable rework. Under a monotone submodular risk-coverage objective, greedy selection of k alerts gets the standard $1 - 1/e$ approximation.

Product and user experience are first-class

Trace work across:

```
UserNeed -> Journey -> Invariant -> Decision -> WorkUnit
          -> Artifact -> Test/Telemetry -> Outcome
```

Who defines UX is a governance choice written in the project constitution. A user-steward role may gather evidence and propose constraints; a product principal owns declared tradeoffs; privacy and express consent can be non-waivable. Subjective disagreement is not a theorem failure. It is a decision with evidence and recourse.

Sharing the durable agentic substrate

Do not serialize an agent as one blob. Separate:

1. Actor build: model requirements, policy digest, skill hashes, tool schemas, effect manifest, eval history.
2. Actor instance: work cursor, commitments, budgets, role epochs, checkpoint.
3. Memory: private raw episodes, curated claims, project memory, public knowledge.
4. Relationship state: user preferences, consent, purposes, confidence, freshness, disclosure policy.
5. Authority: grants, connector credentials, spending, and data-release limits.

The project Harbor stores descriptors and authorized projections, not everyone's full substrate.

Alice can use Bob's capability through distinct modes: signed package transfer, remote invocation in Bob's Harbor, supervised or sealed co-execution, explicit capsule transfer, or a project-memory projection. Discovery, invocation authority, input disclosure, output disclosure, and downstream delegation are separate decisions.

Connector schemas can be shareable; credentials are not. A user relationship is not transferable property. The user or responsible principal controls whether preferences, transcripts, or delegated representation move across the boundary.

Federation boundary: CALM and invariant confluence

Start with a hosted project Harbor providing one authoritative event stream for jointly binding operations, while personal Harbors retain private state. Later, federate with signed event streams, immutable gossip, CRDT catalogs, causal references, and versioned policies.

The coordination boundary is principled:

- Additive evidence, claims, offers, and observations are monotone and can usually converge without coordination.
- Exclusive roles, revocation, finite budgets, unique settlement, acceptance, and replacement are non-monotone and require an authority or consensus point.

This follows the practical lesson of CALM and invariant confluence. It is stronger than pretending everything needs global consensus, and more honest than pretending a peer-to-peer mesh can preserve every invariant during partitions.

Sealed Harbor: Derek and Erin

Derek owns sensitive financial data D . Erin owns a valuable agent system M : weights, prompts, skills, tools, and orchestration. They want $y = f_M(D)$ without Derek receiving M , Erin receiving D , or the cloud operator receiving either.

Call the product **Sealed Harbor** and the technical primitive a **Mutually Confidential Computation Harbor**.

The essential correction is blunt:

Provenance records what happened. Taint records where information is permitted to flow. Neither prevents exfiltration by itself. Security comes from complete effect mediation, constrained release, cryptographic key custody, and an execution substrate neither party can secretly inspect or mutate.

The confidential work order

Both principals sign the same contract before releasing keys:

```
C = <H(data-manifest), H(agent-bundle), H(runtime), H(policy),
    function, tools, recipients, output-schemas, budgets,
    retention, validators, payment, liability>
```

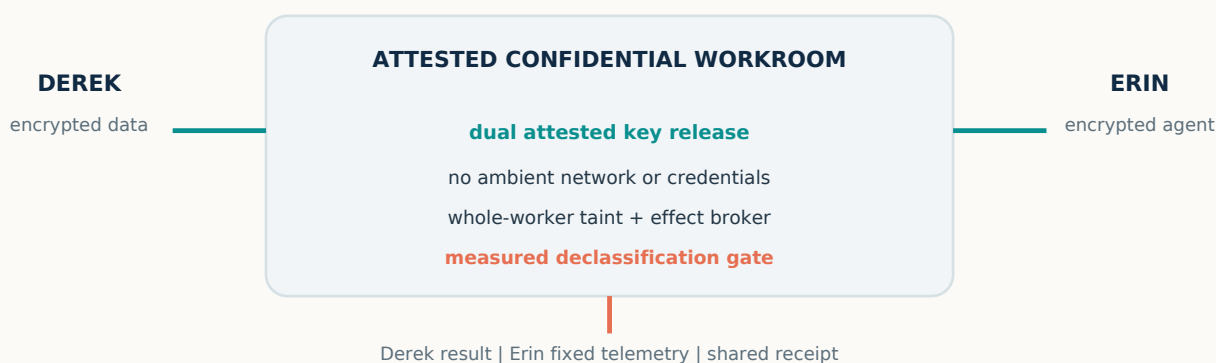
It names the measured runtime and agent bundle, encrypted data inputs, allowed tools, external providers, result and telemetry schemas, output-bit/query/timing budgets, declassification authorities, retention and key destruction, success evidence, dispute procedure, and guarantee. Any change makes a new hash and requires both signatures.

Bring the encrypted agent to the encrypted data

A practical first topology is Derek-controlled cloud infrastructure running a confidential VM:

1. The workload creates an ephemeral job key and fresh attestation evidence binding runtime, policy, worker image, key, nonce, and epoch.
2. Derek and Erin independently verify the attestation.
3. Derek's key broker releases the data key only to the approved measurement.
4. Erin's key broker releases the model/skill key under the same condition.
5. The job runs with no ambient network or credentials.
6. All releases pass through a measured declassifier and an outer Derek-controlled gateway.
7. Both parties receive audience-redacted signed receipts.

IETF RATS gives the attester/verifier/relying-party vocabulary. Confidential VM offerings and attested key-release systems demonstrate the pattern, but expand the trusted computing base to hardware, firmware, cloud control paths, and possibly GPU attestation.



Two fences and no ambient authority

Treat Erin's worker as hostile even if Erin is honest; prompt injection or a dependency can turn it into an exfiltrator. Give it semantic handles only:

```
read(handle, range)
infer(model_handle, prompt_handle)
execute(tool_handle, arguments)
write(result_handle, bytes)
request_declassification(candidate_handle, channel)
```

Use an attested in-guest monitor for labels, handles, and encrypted content, plus a Derek-controlled outer gateway for network, storage, timing, destination, and volume. Direct outbound TLS is forbidden; permitted services terminate at a policy gateway.

An ordinary remote model API becomes another trusted reader if Derek's plaintext is sent to it. Harbor cannot cryptographically repair that. The model must run inside the confidential domain, in another attested confidential provider domain, or through a specialized MPC/FHE protocol.

Coarse information-flow control

Fine-grained token taint through an LLM is not credibly sound. Once the model reads a secret, conservatively taint the whole worker state and every later emission until job destruction.

Use decentralized labels owned by mutually distrustful principals:

```
L(x) = { Derek -> permitted_readers_D,
        Erin  -> permitted_readers_E }

L(derived) = join(labels of all inputs)
```

Ordinary computation can only add restrictions. Removing one owner's clause requires a declassification authority delegated by that owner. Pre-authorize three channels:

- Rich result to Derek.
- Fixed-schema, aggregated or differentially private feedback to Erin.
- Padded public receipt to both.

Robust declassification matters: a malicious Derek prompt must not trick Erin's release rule into revealing the model; a malicious Erin agent must not trick Derek's rule into transmitting data.

The release gate is the real security boundary

No regex, PII scanner, or second LLM can prove that arbitrary prose contains no secret. A secret can be encoded in wording, punctuation, numbers, error messages, output length, timing, or encrypted noise.

If an Erin-visible channel permits b freely chosen bits per job, a malicious worker can exfiltrate up to b bits through it. Across q jobs, the raw capacity is $q \cdot b$ bits. Choosing among s timing slots adds up to $\log_2(s)$ bits per event. Fixed schemas, padding, coarse statuses, query limits, and scheduled release turn an unlimited covert channel into a declared leakage budget.

For row-level aggregate feedback, differential privacy can bound individual influence. Under basic sequential composition, releases consume at most $\sum \epsilon_i$; use a privacy accountant for tighter composition. Differential privacy does not protect the whole dataset or Erin's model IP.

Provenance and receipts

Record a provenance DAG from committed encrypted inputs and agent bundle through activities, candidate result, declassification, and release. Each node records an opaque commitment, parent edges, principal and actor lineage, attested runtime and policy, inherited label, effect/declassification decision, verifier result, nonce, and epoch.

Each party gets a redacted signed receipt:

```
R = Sign_H(H(C), H(attestation), H(inputs), H(M),
           H(output-ciphertext), labels, declassifications,
           counters, nonce, epoch)
```

Use salted or keyed commitments for low-entropy secrets. Anchor Merkle batches independently in both principals' ledgers. The receipt proves what the measured system reported under a policy; it does not prove the answer is true or that no unmodeled side channel existed.

Formal claim worth attempting

Assume TEE confidentiality/integrity/attestation, secure cryptography, complete mediation of modeled effects, correct declassifier enforcement, nonce/epoch replay protection, and exclusion of unmodeled physical and microarchitectural leakage.

Then prove observational noninterference modulo declassification:

```
Obs_E(Exec(D0, M, C)) = Obs_E(Exec(D1, M, C))
```


for datasets equal in public components, except outputs permitted by Derek's release function and declared leakage budget.

The symmetric model-confidentiality property is necessarily weaker because Derek intentionally observes evaluations of M . Unlimited adaptive black-box access can enable model extraction. State confidentiality relative to the contracted query and output interface, not absolute secrecy.

Other tractable properties:

- Label monotonicity: no label weakens without a valid declassification witness.
- Complete mediation: every release has exactly one gate decision.
- Provenance closure: every release traces to committed input, model, runtime, policy, and declassifier nodes.
- Replay safety: each job nonce settles at most once.
- Measurement binding: neither secret key is released for a different runtime or policy.
- Rollback detectability: accepted epochs strictly increase in both external ledgers.

For p principals represented as bitsets, label join/check costs $O(p/w)$ machine words. A provenance graph costs $O(n+e)$ storage; Merkle inclusion is $O(\log n)$; full lineage reconstruction is $O(\text{number of ancestors})$. Output scanning is $O(\text{output length})$ but cannot prove semantic noninterference.

What cannot honestly be promised

- Zero model leakage with unlimited useful adaptive outputs to Derek.
- Zero data leakage with arbitrary natural-language telemetry to Erin.
- Confidentiality from an unapproved external model/tool provider.
- Correct financial advice merely because measured code ran.
- Elimination of all timing, cache, speculative, firmware, or physical channels.
- Availability if either key owner refuses or aborts.
- Literal deletion of information a party already observed.
- Security against colluding hardware vendor, cloud operator, and principal without a stronger multiparty threat model.

MPC and FHE are optional high-assurance backends for narrow fixed computations such as private set intersection, fixed scoring, aggregates, or selected inference. They are not the first implementation for arbitrary tool-using agents.

Theoretical undergirding and honest bounds

The manuscript should label every important statement as one of four things:

1. **Theorem:** follows from a formal model whose assumptions are explicit.
2. **Design invariant:** intended to hold; backed by implementation and tests.
3. **Model-checked property:** proved only for a bounded or abstract program model.
4. **Empirical hypothesis:** needs measurement, calibration, or an experiment.

Do not call a type check or uniqueness constraint a theorem about the world. Conversely, do not undersell properties that can be proved once the boundary is clean.

Mechanism	Formal core	Bound or honest limit
Typed mailbox	Durable log, idempotent reducer, causal order	Dispatch $O(\log S+r)$; queue stable only if admitted $\lambda < \mu$
Exclusive role	Linearizable epoch plus mediated effects	At most one accepted epoch; partitioned stale side must stop
Compaction	Budgeted monotone submodular coverage	NP-hard optimum; greedy $1 - 1/e$ under stated assumptions
Projection sidecar	Capability noninterference, mandatory-ID coverage	Syntactic provenance provable; semantic faithfulness empirical
Effect hypervisor	Complete mediation and capability attenuation	Safety conditional on confinement and enumerated channels
Causal Work Time	Monotone weighted event fold	$O(1)$ update; not a wall clock or productivity truth
Task capsule	Consistent safe-point snapshot plus operation journal	Incremental $O(\delta)$; behavior equivalence impossible
Review fleet	Diverse evidence channels	Independent miss product only under false independence assumption
Lookout	Abstract interpretation and constraint solving	$O(h * \text{affected_edges})$ within finite lattice; semantic completeness impossible
Federation	CALM, CRDT, invariant confluence	Monotone facts coordinate-free; revocation/uniqueness need authority
Confidential IFC	Label lattice and robust declassification	Noninterference only modulo releases and modeled channels
Release channels	Information-theoretic capacity	$q*b$ freely chosen bits across q jobs, before timing channels
Provenance	DAG plus signatures and Merkle commitments	Storage $O(n+e)$; inclusion $O(\log n)$; no truth guarantee

A formal work-unit transition system

The most productive small formal model is not a whole economy. It is one work unit plus principals, roles, capabilities, effects, claims, and receipts.

State:

```
S = <work-units, principals, actor-lineages, role-epochs,
    grants, commitments, operation-journal, artifacts,
    claims, evidence, decisions, settlements>
```

Transitions:

```
propose, authorize, assign-role, send, accept-commitment,
request-effect, admit-effect, record-effect, checkpoint,
assert-claim, attach-evidence, verify, contest,
reassign, revoke, settle
```

Safety properties:

- Every admitted effect has an active grant, correct principal, work unit, role epoch, and resource mapping.
- Each idempotency key produces at most one settled external operation.
- A verified completion has a path to an acceptance policy and an admitted verifier result.

- No child capability is more permissive than its immediate parent.
- A role reassignment invalidates stale epochs at effect brokers.
- Every settlement is funded by the correct principal assets and references a final receipt.

Liveness requires separate assumptions: fair request service, reachable approvers, eventually available verifiers, and bounded partitions where revocation deadlines are promised.

Experiments that matter

The research program should prefer adversarial differential experiments over demos:

- Raw chat versus typed acts: duplicate work, hidden commitments, intervention time.
- Digest versus claim graph: decision accuracy, false belief, time, token cost, source coverage.
- Sidecar attacks: injection, poisoned evidence, omission, conflicting sources, summary recursion.
- Hook mode versus confined mode: bypass success across filesystem, Git, network, credentials, logs, and child processes.
- Cross-provider resurrection: time to useful next action, duplicated effects, obligation loss, final verifier outcome.
- Role failover under kill/partition: stale effects, mailbox continuity, SLO recovery.
- Blind review fleet: seeded real defects, false block rate, joint-miss correlation, post-merge rollback.
- Lookout/Unspider: future incidents only, with precision, recall, Brier calibration, and warning lead time.
- Sealed Harbor red team: canary exfiltration through network, logs, errors, filenames, length, timing, CPU load, and receipt status.
- Confidential performance: plaintext, confidential VM, confidential GPU, and selected MPC/FHE measured separately.

Build sequence

Phase 0: repair the paper before adding machinery

Correct the SDT direction and objective, WAL time claim, exact-key exclusion theorem, fencing omission, identity theorem, float-plan fund ownership, Anchor edge check, Kani proof language, payoff matrix, repeated-game bound, and cross-resource attenuation. Change "roadmap truth" to typed intent projection. Add a one-page table of threat models and assurance levels.

Phase 1: the one-agent receipt

Ship the smallest product that proves the thesis:

- work-unit object and lifecycle;
- typed acceptance policy and completion receipt;
- artifact/evidence graph with why and why-not queries;
- structured live-state projection;
- Portable Task Capsule and same-provider restart;
- operation journal for tool idempotency.

Success means a successor can safely continue after termination and a human can audit why "done" was accepted.

Phase 2: roles, messaging, and confined effects

Add durable role mailboxes, typed acts, lease epochs, fenced tool brokers, isolated workspaces, network/credential mediation, and the projection sidecar. Publish assurance mode per run. Do not claim confinement for hook-only clients.

Phase 3: the review and foresight applications

Build the GitHub review fleet and Lookout/Unspider on the same claim/effect graph. This provides adversarial demand for evidence, role independence, hidden tests, impact analysis, and attention ranking. It also yields the incident corpus needed for CDM skill extraction.

Phase 4: provider-portable resurrection

Define and publish the neutral task-capsule schema. Write provider adapters that reconstruct context but never pretend to restore hidden inference state. Demonstrate Claude-credit exhaustion followed by a different provider resuming without repeated external effects.

Phase 5: hosted project institution

Start with Alice and Bob under one hosted authority for non-monotone operations. Add personal/project memory boundaries, signed skill manifests, remote invocation, user-relationship consent, the coordination graph, typed conflicts, and product-journey traceability. Add Carl only after the two-person semantics are stable.

Phase 6: Sealed Harbor

Begin with a no-network microVM whose rich result only Derek receives. Add dual attested key release, measured policy, whole-worker taint, fixed output channels, signed receipts, and adversarial exfiltration tests. Add confidential GPU and special MPC/FHE kernels later.

Phase 7: economy and underwriting

Only after work receipts and failure rates are real should the market price them. Bind bonds, provider collateral, insurer coverage, and dispute procedures to principal-owned work orders and evidence. Model correlated failure by provider, model family, toolchain, and shared dependency. Include buyer misconduct and evidence poisoning.

Proposed new volume

The papers can be reorganized around one spine:

1. **The Evidence-Bearing Work Unit** - the primitive, lifecycle, and receipt.
2. **The Epistemic Kernel** - claims, evidence, provenance, compaction, and attention.
3. **The Effect Hypervisor** - authority, roles, confinement, and operation safety.
4. **Continuity Without Metaphysics** - task capsules, actor replacement, and inheritance policy.
5. **The Project Institution** - people, agents, user needs, conflicts, and prospective assurance.
6. **The Sealed Harbor** - mutual confidentiality, IFC, attestation, and declassification.
7. **Underwriting Autonomous Work** - principals, receipts, bonds, adjudication, and correlated risk.
8. **Federation** - what can gossip, what needs consensus, and how local authorization crosses boundaries.

The old titles can survive as essays or interludes. But this order makes the causal dependency honest: first define and secure the work; then preserve and share it; only then price it.

Final provocation

The manuscript begins by asking how a swarm becomes a governed society. The stronger question is how uncertain machine work becomes a durable institutional fact.

That move changes everything:

- The agent need not be a person; it must have an attributable lineage.
- The roadmap need not be truth; it must be an amendable intention projection.
- A digest need not be trusted; each claim must carry its warrant.
- A checkpoint need not preserve a mind; it must preserve the work.
- A daemon need not be sovereign; it must completely mediate the effects it claims to control.
- A market need not price souls; it can underwrite receipts.
- A remote Harbor need not merge everyone; it can federate institutions.
- A secure Harbor need not promise magical secrecy; it can make every allowed release explicit and bounded.

The source repo is a projection of the project. The project is a projection of an institution. The institution is made durable by authorized effects, contested claims, evidence, memory, and recourse.

That is the book inside this book.

References and theoretical anchors

1. Saltzer, J. and Schroeder, M. "The Protection of Information in Computer Systems." Proceedings of the IEEE, 1975. Complete mediation, fail-safe defaults, least privilege.
2. Lampson, B. "A Note on the Confinement Problem." Communications of the ACM, 1973.
3. Anderson, J. "Computer Security Technology Planning Study," 1972. Reference-monitor requirements.
4. Denning, D. "A Lattice Model of Secure Information Flow." Communications of the ACM, 1976.
5. Myers, A. and Liskov, B. "A Decentralized Model for Information Flow Control." SOSP, 1997. <https://www.cs.cornell.edu/andru/papers/iflow-sosp97/paper.html>
6. Zdancewic, S. and Myers, A. "Robust Declassification." CSFW, 2001.
7. Chandy, K. and Lamport, L. "Distributed Snapshots: Determining Global States of Distributed Systems." ACM TOCS, 1985.
8. Lamport, L. "Time, Clocks, and the Ordering of Events in a Distributed System." Communications of the ACM, 1978.
9. Hellerstein, J. and Alvaro, P. "Keeping CALM: When Distributed Consistency Is Easy." 2020.
10. Bailis, P. et al. "Coordination Avoidance in Database Systems." VLDB, 2014. Invariant confluence.
11. FIPA. "Agent Communication Language Specifications." <https://www.fipa.org/specs/fipa00003/OC00003A>
12. Agha, G. Actors: A Model of Concurrent Computation in Distributed Systems. MIT Press, 1986.

13. Klein, G., Calderwood, R., and Macgregor, D. "Critical Decision Method for Eliciting Knowledge." IEEE TSMC, 1989. <https://doi.org/10.1109/21.31053>
14. van der Aalst, W. et al. "Workflow Mining: Discovering Process Models from Event Logs." IEEE TKDE, 2004.
15. Lin, H. and Bilmes, J. "A Class of Submodular Functions for Document Summarization." ACL, 2011. <https://aclanthology.org/P11-1052/>
16. Nemhauser, G., Wolsey, L., and Fisher, M. "An Analysis of Approximations for Maximizing Submodular Set Functions." Mathematical Programming, 1978.
17. Cheney, J., Chiticariu, L., and Tan, W. "Provenance in Databases: Why, How, and Where." Foundations and Trends in Databases, 2009.
18. W3C. "PROV-O: The PROV Ontology." <https://www.w3.org/TR/prov-o/>
19. IETF. RFC 9334, "Remote Attestation procedureS (RATS) Architecture," 2023. <https://www.rfc-editor.org/rfc/rfc9334.html>
20. NIST. SP 800-226, "Guidelines for Evaluating Differential Privacy Guarantees," 2025. <https://doi.org/10.6028/NIST.SP.800-226>
21. NIST. Privacy-Enhancing Cryptography project. <https://csrc.nist.gov/Projects/pec>
22. Tramer, F. et al. "Stealing Machine Learning Models via Prediction APIs." USENIX Security, 2016.
23. in-toto. Software supply-chain integrity framework. <https://in-toto.io/>
24. SLSA. Provenance specification. <https://slsa.dev/spec/v1.2/provenance>
25. SQLite. WAL auto-checkpoint API. https://sqlite.org/c3ref/wal_autocheckpoint.html
26. Linux kernel. Landlock userspace API. <https://www.kernel.org/doc/html/latest/userspace-api/landlock.html>
27. gVisor. Architecture Guide. https://gvisor.dev/docs/architecture_guide/intro/
28. Firecracker. MicroVM design. <https://github.com/firecracker-microvm/firecracker/blob/main/docs/design.md>
29. Google. "Zanzibar: Google's Consistent, Global Authorization System." USENIX ATC, 2019.
30. OpenAI. Conversations API. <https://platform.openai.com/docs/api-reference/conversations/>
31. Google. Gemini thought signatures and state handling. <https://ai.google.dev/gemini-api/docs/thought-signatures>
32. The Kani Rust Verifier. Proof harness reference. <https://model-checking.github.io/kani/reference/attributes.html>
33. SPIFFE. Trust-domain federation specification. https://spiffe.io/docs/latest/spiffe-specs/spiffe_federation/
34. IETF. RFC 8725, JWT Best Current Practices; RFC 8693, OAuth Token Exchange; RFC 9449, DPoP.
35. The attached collected volume, The Harbor, the Person, and the Economy: The Collected Coordination Papers, August 2026.